ARCHITECTURE DOCUMENT

Group 5: Smart Farming
Software Architecture (2026–2027)
University of Groningen



Students:
Divyashri Ravichandran (S6596940)
Maria Voicu (S5585635)
Anh Khoi Nguyen Ho (S6867162)
Ivan De Simone (S6817521)
Joël Kiers (S4042204)

Last build: October 27, 2026

VERSION 0.8

Revision History

Contributors:

- dr: Divyashri Ravichandran
- mv: Maria Voicu
- jk: Joël Kiers
- aknh: Anh Khoi Nguyen Ho
- ids: Ivan De Simone
- all: Everyone

Date	Revision	Author	Description
2026-09-01	0.0	dr	Set up the document
2026-09-03	0.1	mv	Drafted Target Audience & SWOT analysis sections
		dr	Drafted unique selling points & competitors sections
		jk	Drafted stakeholders and concerns
		aknh	Drafted Systems Context and Financial Model
		ids	Drafted use-case diagram and list
2026-09-10	0.2	jk	Updated stakeholders based on design workout 1
		mv	Drafted Business Rationale & updated Target Audience and Business Vision
		dr	Updated competitors, drafted Domain & Business models
		aknh	Refined Context and Solution Description narrowing down project scope
		aknh	Added product development business strategy in Business Model & updated Financial Model
		ids	Refined use case diagram and tables
2026-09-14	0.3	ids	Drafted business roadmap
		dr	Added assumptions and descriptions for the figure 2.1 & 2.2
2026-09-15		ids	Refined financial model
		mv	Added product description and non-functional requirements
2026-09-16		ids	Added technology roadmap
		dr	Added functional requirements & high-level design decisions
2026-09-17		ids	Sketched C4 context and containers views
		jk	Added evolution requirements and updated ISO/IEC 25010 description
		aknh	Added Architectural Vision
		aknh	Refined Use-case scenarios 1 to 3 & Fixed sections 3.1 to 3.3 formating
		aknh	Added three risks and Risk Assessment diagram
2026-09-18		dr	Refined figure 2.1 & 2.2
		ids	Refined C4 models and descriptions
		aknh	Refined Use-case scenario 4 & Updated Risk Assessment with 3 additional risks
2026-09-23	0.4	dr	Drafted 6.1 hardware components section
2026-09-24		jk	Added 5.3 verification based on section 5 and existing UC's
		mv	Drafted section 7.1
		aknh	Refined Architecture Vision
		aknh	Added details for Risk Table
		aknh	Modified UC5 and UC6 & Added failing scenario for UC1
		ids	Refined tech roadmap, section 5.1 and 5.2

Date	Revision	Author	Description
2026-10-01	0.5	dr	Added Hardware Decisions (HD-01, HD-02)
		aknh	Added hyperlinks for Functional Requirements in use-case scenarios table
		ids	Detailed elaborated model and technologies rationale
		mv	Added Hardware Decisions (HD-03, HD-04)
2026-10-02	0.6	ids	Added logo and alternatives model subsection
2026-10-06		dr	Added Hardware Decisions (HD-05, HD-06)
		mv	Added Hardware Decisions (HD-07, HD-08)
2026-10-07		ids	Drafted regulator backend logical view
		ids	Drafted deployment view
2026-10-08		mv	Drafted farm backend logical view
		dr	Drafted user interface logical view
		ids	Detailed sections 5.1 and 5.2
2026-10-13	0.7	ids	Update business roadmap
		ids	Reworked financial model
		dr	Moved hardware decisions to appendix
		dr	Drafted field gateway logical view
		ids	Refined tech roadmap
		mv	Refined business vision & product description
2026-10-14		dr	Added software design decisions
2026-10-15		jk	Added architecture evaluation
		mv	Added database schema
		ids	Refine deployment view and draft process view
		aknh	Refined Hardware Decision HD-1 & Added Hardware Overview description and diagram
2026-10-16		aknh	Refined Use cases UC-2, UC-3 and UC-5 & Updated Use case diagram
		aknh	Updated Risk Assessment R4 and R5
2026-10-22	0.8	mv	Added chapter 9
		dr	Added prototype description
		ids	Update deployment and process diagrams
2026-10-26		ids	Refined process view
2026-10-27		ids	Refined alternative models

Contents

List of Figures 7

List of Tables 8

1 System Context 9
- 1.1 Context . . . 9
- 1.2 Solution Description . . . 9

2 Business Information 10
- 2.1 Business Vision . . . 10
  - 2.1.1 SWOT Analysis . . . 10
- 2.2 Business Rationale . . . 11
- 2.3 Product Description . . . 12
- 2.4 Target Audience . . . 12
- 2.5 Business/Domain Model . . . 13
  - 2.5.1 Business Model . . . 13
  - 2.5.2 Domain Model . . . 14
- 2.6 Business Roadmap . . . 15
- 2.7 Financial Model . . . 15
  - 2.7.1 Architecture Cost . . . 16
  - 2.7.2 Consultancy Cost . . . 16
  - 2.7.3 Development Cost . . . 16
  - 2.7.4 Funding . . . 16
  - 2.7.5 Hardware Cost . . . 16
  - 2.7.6 Product Pricing . . . 17
- 2.8 Competitors . . . 17
  - 2.8.1 Kilimo . . . 18
  - 2.8.2 SWAN Systems . . . 18
  - 2.8.3 Agrow Analytics . . . 18

3 Requirements 19
- 3.1 Architectural Vision . . . 19
- 3.2 Stakeholders and their Concerns . . . 20
- 3.3 Stories and Use cases . . . 22
- 3.4 Functional Requirements . . . 28
- 3.5 Commercial Non-Functional Requirements . . . 28
- 3.6 Technical Non-Functional Requirements . . . 29
- 3.7 Evolution Requirements . . . 31
- 3.8 Risk Assessment . . . 32

4 Analysis **33**
4.1 Assumptions . . . 33
4.2 Technology Roadmap . . . 33
4.3 High-level Design Decision . . . 34

5 System Architecture **37**
5.1 Initial Model . . . 37
5.2 Elaborated Model . . . 37
5.2.1 Technologies rationale . . . 39
5.3 Alternative models . . . 39
5.3.1 Cloud-based service . . . 39
5.3.2 Direct field communication . . . 40
5.3.3 In loco weather sensors . . . 40
5.4 Verification . . . 40
5.4.1 Performance . . . 40
5.4.2 Capacity . . . 41
5.4.3 Reliability and Availability . . . 41

6 Hardware Architecture **42**
6.1 Hardware Decisions . . . 42
6.2 Hardware Overview . . . 42

7 Software Architecture **44**
7.1 Architecturally Relevant Drivers . . . 44
7.2 Logical View . . . 44
7.2.1 Farm Backend Components . . . 45
7.2.2 Regulator Backend Components . . . 45
7.2.3 Local Database Components . . . 46
7.2.4 User Interface Components . . . 47
7.2.5 Field Gateway Components . . . 48
7.3 Implementation View . . . 49
7.3.1 Repository Structure . . . 49
7.3.2 Package diagram . . . 49
7.4 Process View . . . 50
7.4.1 System Operations . . . 50
7.4.2 Farm Backend Operations . . . 51
7.5 Deployment View . . . 52
7.5.1 Cloud Storage . . . 52
7.5.2 Local Kubernetes Cluster . . . 52
7.5.3 Physical Devices . . . 52
7.6 Software Design Decisions . . . 53

8 Architecture Evaluation **57**
8.1 Requirements Verification . . . 57
8.2 Architecture Trade-off Analysis Method . . . 58
8.2.1 Utility tree . . . 58
8.2.2 Scenario Analysis . . . 59
8.3 Prototype Description . . . 59

9 System Evolution **61**
9.1 Future extensions . . . 61

9.1.1 Machine learning model for irrigation decisions . . . 61
9.1.2 Awareness of available energy resources . . . 62
9.1.3 Crop health and yield insights . . . 62
9.1.4 Multiple farm locations per owner . . . 63
9.2 Versions . . . 63

References **64**

A Architecture Decision Records **67**
A.1 Hardware Decisions . . . 68
A.1.1 Controller Architecture . . . 68
A.1.2 Soil Moisture Sensor . . . 69
A.1.3 Flow meter . . . 70
A.1.4 Solenoid Valve . . . 71
A.1.5 Connectivity . . . 72
A.1.6 Power Supply . . . 73
A.1.7 Water Pump . . . 74
A.2 Title . . . 75
A.3 Title . . . 76

B Time Tracking **77**
B.1 Week 1 . . . 77
B.2 Week 2 . . . 78
B.3 Week 3 . . . 79
B.4 Week 4 . . . 80
B.5 Week 5 . . . 81
B.6 Week 6 . . . 81
B.7 Week 7 . . . 82
B.8 Week 8 . . . 83
B.9 Week 9 . . . 83

List of Figures

2.1 Business Model diagram 13
2.2 Domain Model diagram 14

3.1 Architectural Vision diagram 20
3.2 Irrigation Use Case Diagram 22
3.3 Report Export Use Case Diagram 23

5.1 System Context view 37
5.2 Containers view 38

6.1 Hardware Overview 43

7.1 Farm Backend's Logical View diagram 45
7.2 Regulator Backend's Logical View 46
7.3 Diagram of the local database schema 46
7.4 User Interface's Logical View 48
7.5 Field Gateway Logical View 48
7.6 Directory structure of system 49
7.7 Package diagram of the system modules and shared artifacts 50
7.8 Sequence diagram - System Operations 51
7.9 Sequence diagram - Farm Backend Operations 52
7.10 Deployment diagram 53

List of Tables

2.1 SWOT analysis . . . 11
2.2 Business roadmap . . . 15
2.3 Average hardware prices . . . 17

3.1 List of stakeholders . . . 21
3.2 UC1 - Collect data . . . 24
3.3 UC2 - Analyze data . . . 25
3.4 UC3 - Track water use . . . 25
3.5 UC4 - Export on-demand water use report . . . 26
3.6 UC5 - Implement irrigation decision . . . 26
3.7 UC6 - Authenticate user . . . 27
3.8 Risk Table . . . 32

4.1 Technology roadmap . . . 34
4.2 ADR - Data Storage . . . 34
4.3 ADR - External Weather Data Integration . . . 35
4.4 ADR - Security and Access Control . . . 35
4.5 ADR - Field Gateway Network Architecture . . . 36

6.1 A brief overview of the hardware decisions. . . . 42

7.1 Decision - Architectural Style . . . 53
7.2 Decision - Communication . . . 54
7.3 Decision - Data Transfer Pattern . . . 55

8.1 Verification of architecturally significant requirements . . . 57
8.2 Architecture prototype verification . . . 58
8.3 ATAM utility tree (precursor) . . . 59

9.1 Planned versions of the Smart Irrigation system . . . 63

A.1 Decision - Controller Architecture . . . 68
A.2 Decision - Soil Moisture Sensor . . . 69
A.3 Decision - Flow meter . . . 70
A.4 Decision - Solenoid Valve . . . 71
A.5 Decision - Connectivity . . . 72
A.6 Decision - Power Supply . . . 73
A.7 Decision - Water pump . . . 74

1 — System Context

In this chapter, we present the context of the system and provide a brief description of the proposed solution.

1.1 Context

The world population is expected to grow rapidly toward 9.8 billion by 2050. Faced with this situation, in 2022, the late deputy director of the Center on Food Security and the Environment at Stanford University, Walter P. Falcon, and his team estimated that agricultural output must increase by 50% to 60% compared to 2019 baselines [40].

Additionally, natural disasters and climate change have worsened the situation in recent years by degrading farming conditions worldwide—not to mention the increasing obsolescence of traditional farming methods due to environmental impacts and skyrocketing input costs for seeds and fuel.

Mediterranean agriculture is particularly exposed due to critical seasonal water scarcity and poor soil conditions. Low organic matter and soil crusting caused by heavy rains, combined with high alkaline pH (7.5–8.5), have led to restricted water infiltration and poor water-holding capacity. Yet, main Mediterranean crops such as wheat and grapevines are naturally water-demanding.

Therefore, **Smart Farming** is becoming an operational necessity across the Mediterranean.

Definition of Smart Farming – ISO/TMB Strategic Advisory Group on Smart Farming ("SAG-SF") Final Report, January 2023:

> *Smart Farming is data-driven, principled decision-making in agricultural and food value chains occurring as multi-objective optimization in the context of global volatility, uncertainty, complexity, and ambiguity.*

1.2 Solution Description

Our team of software architects proposes an energy-efficient Smart Irrigation coordination system built for forward compatibility with future third-party hardware. By integrating popular sensors and sprinklers with predictive weather analytics, the system optimizes crop health and pH control while reducing water waste, environmental impact of fertilization and cutting input costs. Built for simplicity, farmers can easily register third-party equipment and customize delivery by crop type without technical hassle. Our flexible software architecture guarantees long-term maintainability and effortless updates across multi-vendor ecosystems.

2 — Business Information

The chapter outlines the business aspects of our product. It begins with a comprehensive overview of the business vision and the rationale behind it, followed by in-depth descriptions of the envisioned product and the target customers. It then presents the proposed domain and financial models, a roadmap of key milestones, and concludes with a survey of existing competition.

2.1 Business Vision

With drought events and water scarcity becoming increasingly frequent across European countries in the Mediterranean region, smart irrigation is one of many solutions that can be employed to help mitigate these issues.

Our product monitors farmland soil characteristics such as humidity, temperature, salinity, and pH, while also accounting for climate-related factors, namely weather forecasts and evapotranspiration data. When predicting the resources needed to irrigate a certain parcel, specific plant needs are also considered. In doing so, we aim to improve the precision of pre-existing automatic irrigation systems, reducing overall water wastage in the agriculture. To achieve this goal, the proposed system focuses on the following three primary aspects:

- **Data collection**: Underground IoT sensors continuously track soil properties (humidity, temperature, salinity, and pH). These recordings are coupled with third-party weather and evapotranspiration data, as well as information entered by the user to estimate precise water quantities at the crop level.
- **Analysis and decision making**: Once all required information has been gathered, the system uses a decision making model to produce irrigation decisions.
- **Automatic/Manual implementation**: The system forwards pending irrigation decisions to on-field devices, which then deliver the water automatically. Alternatively, manual irrigation is possible through user-defined decisions.
- **Insight visualization**: Users are able to review water and fertilizer usage trends through an intuitive dashboard in the UI.

2.1.1 SWOT Analysis

The envisioned system has been assessed in accordance with SWOT criteria [42]. Table 2.1 illustrates the findings of this analysis, evaluating internal factors such as Strengths and Weaknesses, as well as external ones, specifically Opportunities and Threats.

Strengths (Internal, Positive)	**Weaknesses** (Internal, Negative)
• Compatible with pre-existing farm device infrastructure • Sustainable practices through efficient water resource management • Fully automated process	• Hardware is not produced in-house • Our team has low-experience with the agriculture sector
Opportunities (External, Positive)	**Threats** (External, Negative)
• Smart Farming is an emerging topic • Drought events are expected to appear more frequently in the coming years [10] • Attracts clients with an interest in water management	• Competitive market in the future, as Smart Farming practices continue to rise in popularity [12]

Table 2.1: SWOT analysis

Thus, selling points for this software can be summarized as follows:

- **Sustainable yet profitable**: With water getting more scarce in the Mediterranean region, watering crops can become expensive. Prediction-based irrigation is cheaper for the farm owners, while also wasting less resources.
- **Modularity**: The proposed system is capable of supporting a range of off-the-shelf sensors that the user may already use.
- **Full automation**: The entire monitoring, analysis and irrigation process is fully automated in accordance with predictions made by the system.
- **Workload reduction**: The envisioned system reduces the amount of tasks and factors farm workers have to specifically track in order to maintain crop production above market thresholds.
- **Improved crop yield**: By tailoring water delivery to each plant's specific needs, the system helps crops grow under more optimal conditions, leading to higher and more consistent yields.

2.2 Business Rationale

The core motivation behind our product is to optimize crop irrigation throughout the European Mediterranean region. As drought events grow more frequent and temperatures continue to rise, existing infrastructures are increasingly unable to adapt to these conditions. Moreover, as water scarcity becomes more prevalent, it is vital that major resource consumers, such as the agriculture sector, transition to more environmentally sustainable prac-

tices. Our software addresses this directly, offering a fully automated, modular system that optimizes water resource management, while supporting more consistent crop yields, and balancing the workload of farm workers.

2.3 Product Description

As sustainable farming practices have become more popular in recent years, the precision irrigation market has also grown considerably. Competitors in this field rely on the continuous monitoring of farmland soil properties, using devices such as underground IoT sensors, water meters, and automatic sprinklers (some examples are discussed in Section 2.8). However, most systems are built on proprietary hardware, effectively vendor-locking farmers. Designing a product that is both hardware- and vendor-independent is a goal we aim to achieve.

Our product is a smart irrigation system designed to reduce water wastage across European countries in the Mediterranean, starting with Italy as a pilot region. It combines up-to-date soil and climate monitoring with automated irrigation, optimizing water and fertilizer use for grape, olive and wheat farms. Moreover, farmers are offered actionable insights into resource usage trends per crop type.

To measure farm soil properties (humidity, temperature, salinity, and pH), the system can be connected to IoT sensors and water meters. As mentioned, its design is highly flexible, meaning it is compatible with a wide variety of device providers, encouraging farm owners to reuse pre-existing equipment. Third-party data, namely weather and evapotranspiration information, are pulled by the system from dedicated APIs. This information is compiled and stored to be used for later analysis.

Once the system gathers all required data, an integrated decision-making model is used to compute irrigation decisions. The system then schedules these decisions for on-field implementation, fully automating the irrigation process. Alternatively, users can manually deploy water by defining their own commands, which take priority over system-generated ones. This approach contributes to a more consistent long term crop yield.

Upon completion, farm owners and workers can review water and fertilizer usage trends per plant type via a dashboard in the system's user interface. These insights span both short-term and long-term time periods, supporting better decision-making. The system can also generate reports of these trends for sharing with external parties who do not have access to the system directly.

2.4 Target Audience

The proposed software aims to mitigate water scarcity in the Mediterranean region of Europe by optimizing water use in the agricultural sector, particularly for wheat, olive and grape farms. The main target of the initial stage is Italy, where in 2024 over 91% of agricultural holdings reported irrigation-related difficulties [37], with plans to extend the interested area to Spain, Portugal and Greece in the future. To achieve our goal, two different types of customers are identified:

- **Large to medium-sized companies (50+ hectares of utilised agricultural area)** [36]: Agriculture businesses with significant production outputs seeking to optimize irrigation through more sustainable practices
- **Single-owned farms**: Farm owners seeking to cut down water wastage, who can afford such a system

2.5 Business/Domain Model

2.5.1 Business Model

Product Development Business Strategy

- **Capital Efficiency & Hardware Agnosticism**: Minimizes upfront hardware development costs by leveraging Commercial Off-The-Shelf (COTS) components across multiple vendors, ensuring broad software compatibility across standard controller hubs.
- **Vendor Decoupling**: Eliminates costly third-party integration fees by restricting primary hardware support to open or well-documented communication protocols.
- **Tiered Hardware Validation Strategy**: Optimizes operational testing through a two-phase approach:
  - **Legacy Hardware**: Uses refurbished older-generation devices internally to verify backward and cross-generational software compatibility at low cost
  - **Next-Gen Hardware**: Engages farmers in a structured pilot program to test new hardware adapters in real-world scenarios in exchange for promotional rates or service discounts
- **Feedback-Driven ROI**: Directly channels field-test telemetry into iterative software improvements to increase user retention and product-market fit.

Stakeholder Relation Business Model

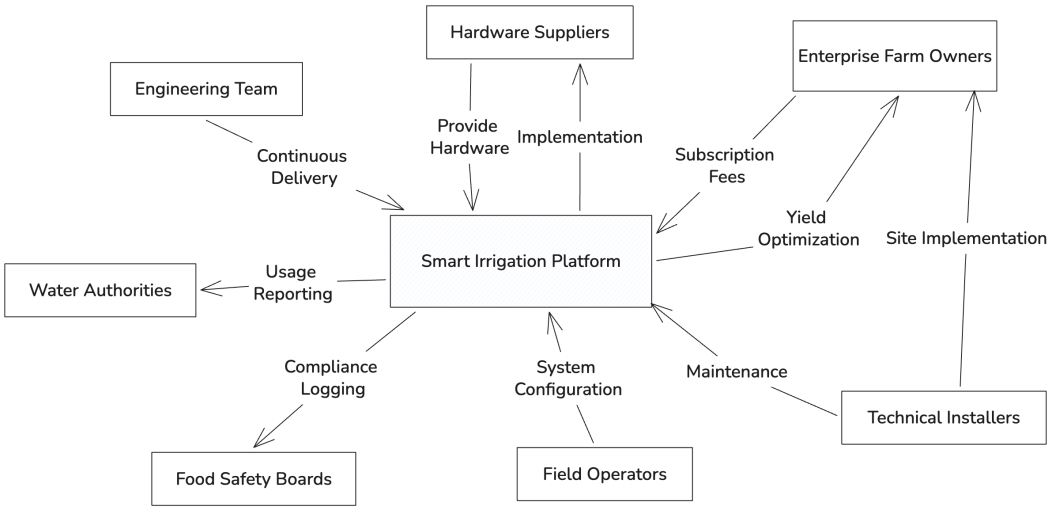


Figure 2.1: Business Model

As illustrated in Figure 2.1, the interactions between the system and its primary stakeholders are defined as follows:

- **Enterprise Farm Owners**: Pay Subscription Fees to the Smart Irrigation Platform and receive Yield Optimization insights in return. They also receive Site Implementation services performed by Technical Installers.
- **Field Operators**: Provide System Configuration to the Smart Irrigation Platform to manage field operations.

- **Technical Installers**: Handle the Site Implementation for Enterprise Farm Owners.
- **Engineering Team**: Performs Continuous Delivery updates and maintenance on the Smart Irrigation Platform.
- **Hardware Suppliers**: Provide Hardware Integration capabilities to the Smart Irrigation Platform.
- **Water Authorities**: Receive Usage Reporting sent from the Smart Irrigation Platform.
- **Food Safety Boards**: Receive Compliance Logging generated by the Smart Irrigation Platform.

2.5.2 Domain Model

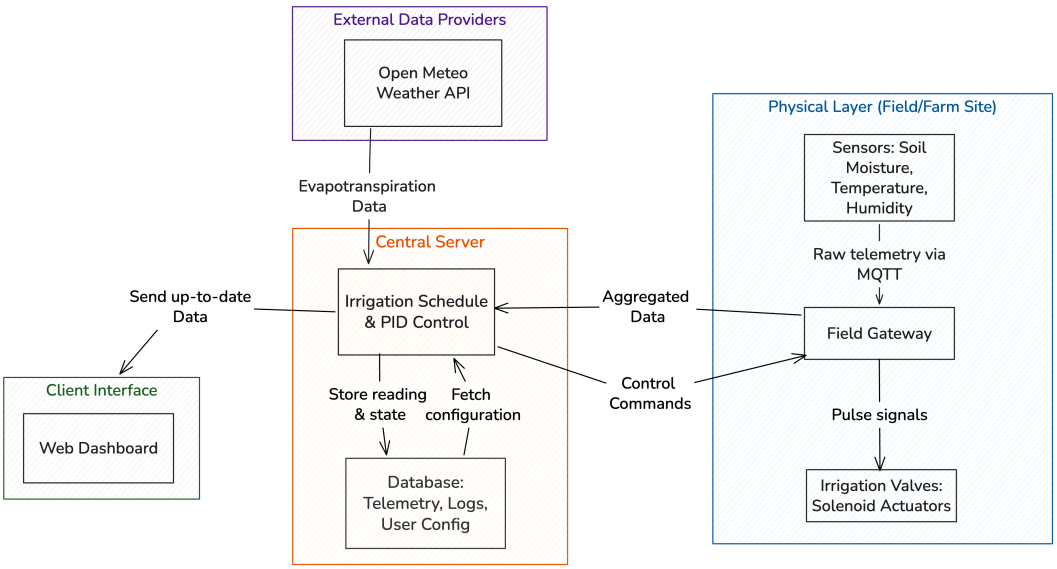


Figure 2.2: Domain Model

The components of the system and their technical interactions, depicted in Figure 2.2 are organized into three main operational domains:

Physical Layer

- **Sensors (Soil Moisture, Temperature, Humidity)**: Transmit Raw telemetry via MQTT [26] to the Field Gateway.
- **Field Gateway**: Aggregates incoming sensor telemetry and sends Aggregated Data to the central engine. It receives Control Commands from the central engine and outputs Pulse signals to drive the actuators.
- **Irrigation Valves (Solenoid Actuators)**: Receive Pulse signals from the Field Gateway to open or close valves.

Central Server

- **Irrigation Scheduler & PID Control**:
  - Receives Aggregated Data from the Field Gateway and sends back Control Commands.

  - Executes Store reading & state operations to update the Database and executes Fetch configuration queries from it.
  - Outputs Send Up-to-date Data streams to the Client Interface.
- **Database (Telemetry, Logs, User Config)**: Stores system telemetry and logs while serving user configurations to the Irrigation Logic Engine.

External Data Providers & Client Interfaces

- **Open-Meteo Weather API**: Sends Evapotranspiration and weather forecast data to the system.
- **Client Interface (Web Dashboard)**: Displays analytics and up-to-date status information.

2.6 Business Roadmap

In the early stages of deployment, European countries already facing water scarcity and droughts, specifically in the Mediterranean region, will be targeted first. For them, our product will be not only a business optimization, but, more importantly, an environmental necessity.

At first, Italy will serve as a pilot country, considering the previously mentioned irrigation-related issues, the massive investments in agro-irrigation system resilience [19], and the already existing smart-irrigation systems [30] [34] [7]. After establishing a strong presence in the pilot country, we will expand our scope to other European Mediterranean countries facing similar conditions, namely Spain, Portugal and Greece. In doing so, different laws and cultural approaches should be taken into account. As the system matures over time, our interested area will be extended to all European countries, and subsequently, to other non-European countries confronting comparable water insufficiency issues.

Estimating approximately one year for the development of the system, an overview of the roadmap is as follows in Table 2.2.

Table 2.2: Business roadmap

Period	Area	Milestone
2028 Q1	Italy	Launch system in Italy
2028 Q3	European Mediterranean	Launch system in Spain, Portugal and Greece
2029 Q3	Europe	Launch system across Europe
2030 Q3	North Africa	Launch system outside of Europe

2.7 Financial Model

This section describes at high-level the costs for the design, development, and hardware required by the system, as well as the funding options available. In addition, it covers the deployment and maintenance pricing for the final customers.

2.7.1 Architecture Cost

The architecture document is produced by a team of five software architects, working on the project for an average time of 120 hours each. The work of one architect is valued at 100 euros per hour. [32]

Total architecture cost = 5 x 120 x 100 € = **60.000 €**

2.7.2 Consultancy Cost

Since the team producing the design document for the system has low experience in the agricultural sector, it is required the presence of an agricultural consultant, in order to drive towards correct and feasible decisions, for an estimated time of 64 hours. The work of one consultant is valued at 80 euros per hour.

Total consultancy cost = 1 x 64 x 80 € = **5.120 €**

2.7.3 Development Cost

A software system of this magnitude is expected to be carried out by a software development team composed of six developers. It is estimated that every developer will spend 30 hours a week working on the project, for a total of 40 weeks. The work of one developer is valued at 75 euros per hour. [32]

Total development cost = 6 x 30 x 40 x 75 € = **540.000 €**

2.7.4 Funding

The design and development costs for the Smart Irrigation system could be partially, if not completely, covered by EU funding regarding the water waste management and innovation. On this matter, two funding programs are identified:

HORIZON-CL6-2027-02-FARM2FORK-01 Increasing the resilience of agriculture in water and nutrient-scarce environments through digital innovations [8]

PRIMA Integrated and sustainable management of water for arid and semi-arid Mediterranean areas [33]

2.7.5 Hardware Cost

First, it needs to be clarified that the investment for hardware parts is in no way linked to any source of revenue or expenses for our company, as it is a potential expenditure for the farm itself. The following estimations will have to be considered in the context of the specific farm, taking into account which devices are already installed and which ones need to be purchased. Finally, the cost of hardware components cannot be predetermined completely upfront, due to adaptability to the farmland size and characteristics. For this reason, the average prices for sensors and other necessary hardware are provided in Table 2.3, and will be considered within the scope of a medium-large farm.

Table 2.3: Average hardware prices

Hardware type	Average price
Soil moisture sensor	150 €
IoT gateway	400 €
Irrigation valve	375 €
Water flow meter	1100 €

According to the aforementioned premises, an estimated set of devices, required to equip a 50 hectare farm lacking any kind of smart hardware, would include roughly: one soil moisture sensor per hectare, one irrigation valve every 5 hectare, two IoT gateways (plus two backup ones) and a single water flow meter. An overview of the costs would be as follows:

- 50 soil moisture sensors = 7.500 €
- 10 irrigation valves = 3.750 €
- 4 field gateways = 1.600 €
- 1 water flow meters = 1.100 €

In addition to these components, a central server unit is needed in order to process and memorize the data. Indicative minimum requirements are a modern 8–16 core server CPU, 32 gigabytes of RAM, 4 terabytes of usable disk space, and network connectivity, for an estimated cost of 3.000 €.

Total hardware cost = 7.500 € + 3.750 € + 1.600 € + 1.100 € + 3.000 € = **16.950 €**

2.7.6 Product Pricing

Revenue from our solution comes from the initial selling price as well as ongoing maintenance and post-deployment services. To accommodate this, our team has established two plans: the first is a pay-as-you-go structure, while the second is based on continuous annual subscriptions. The former gives the client flexibility, while the latter ensures stable funds for future development.

Both plans will provide substitution of malfunctioning components, software updates, and customer assistance. The initial selling price, including technical installation and professional education, will be the same, regardless of which option the consumer chooses, and has been determined as 2.000 €.

The price for the pay-as-you-go plan will be calculated based on the type of operation requested, on top of which a 20% profit margin is counted. The price for the annual subscription plan will be fixed at 40 € per hectare in the beginning, with the possibility of being adjusted in the future.

2.8 Competitors

The market already contains platforms that combine weather, satellite, soil, and irrigation data to improve agricultural water use. Competitors also use scheduling, satellite imagery and AI-driven decision models. We take our competitors' approach into consideration when

making architectural decisions. Existing solutions tell us where our specifications should stick to convention and where it is better to diverge.

2.8.1 Kilimo

Kilimo is a decision platform that uses meteorological information, satellite imagery, and field data to estimate crop water requirements. Its system calculates local evaporation and transpiration rates and uses crop coefficients along with this information to provide local irrigation recommendations. Besides supporting irrigation decisions, Kilimo also focuses on measuring and verifying water savings at watershed level.

Kilimo shows us the power of combining satellite, weather and soil information to make irrigation decisions instead of just relying on individual sensors. However, their focus is mainly on making recommendations and not on increasing automation by controlling valves or actuators. Our system would make similar irrigation decisions, but go one step beyond by handling irrigation itself automatically and validating the results.

2.8.2 SWAN Systems

SWAN Systems is another platform that is aimed at irrigation. It combines soil models with weather information, measured soil moisture and satellite imagery in order to make irrigation plans. The platform is web-based and keeps track of water consumption and can also be connected to irrigation automation systems.

The most relevant characteristic of SWAN Systems is their hardware independence. Existing hardware and equipment can be connected to the platform without requiring replacement from specific vendors. Its dashboard supports water budgeting, soil moisture predictions and even crop health information based on satellite imagery, provided the according hardware is integrated. What we can learn is that a hardware agnostic integration strategy requires well defined device interfaces and must avoid dependencies on individual sensors.

2.8.3 Agrow Analytics

Agrow Analytics is a Spanish platform that handles water management and precision irrigation. It is relevant to our intended Mediterranean market and combines information about crops, the environment and previous irrigation to recommend where, when and how much to irrigate. The platform can use the forecasts of crop water consumption and rainfall to make irrigation schedules. The platform can be integrated with existing irrigation controllers, valves and actuators. This way, its recommendations can be translated into automatically-executed watering schedules with approval of an operator.

Agrow could be considered our most direct competitor due to our overlapping target audience and similar irrigation automation strategy. In order to effectively differentiate from them, we need to ensure that besides automation, our system is compatible with many vendors and hardware independent.

3 — Requirements

In this chapter, stakeholders and their concerns are identified. Next, a list of use cases addressing these concerns is provided. The subsequent sections cover the requirements of the system, categorized in functional, commercial non-functional, technical non-functional, and evolution requirements, prioritizing them by using the MoSCoW method [41]. Finally, the risk assessment is described.

3.1 Architectural Vision

Figure 3.1 depicts the envisioned architecture of the system. The vital component is the Controller Hub, whose tasks are collecting and processing data, interacting with the Local Database, and exposing endpoints for the users to access information. Not less important, Smart Farming Devices within the farm will send measurements to the Controller Hub and will receive back irrigation decisions in the form of instructions. Third-party Weather API are also used to fetch data to support decisions. Finally, Regulators and Farmers will access system information respectively through a Remote API and a Local GUI.

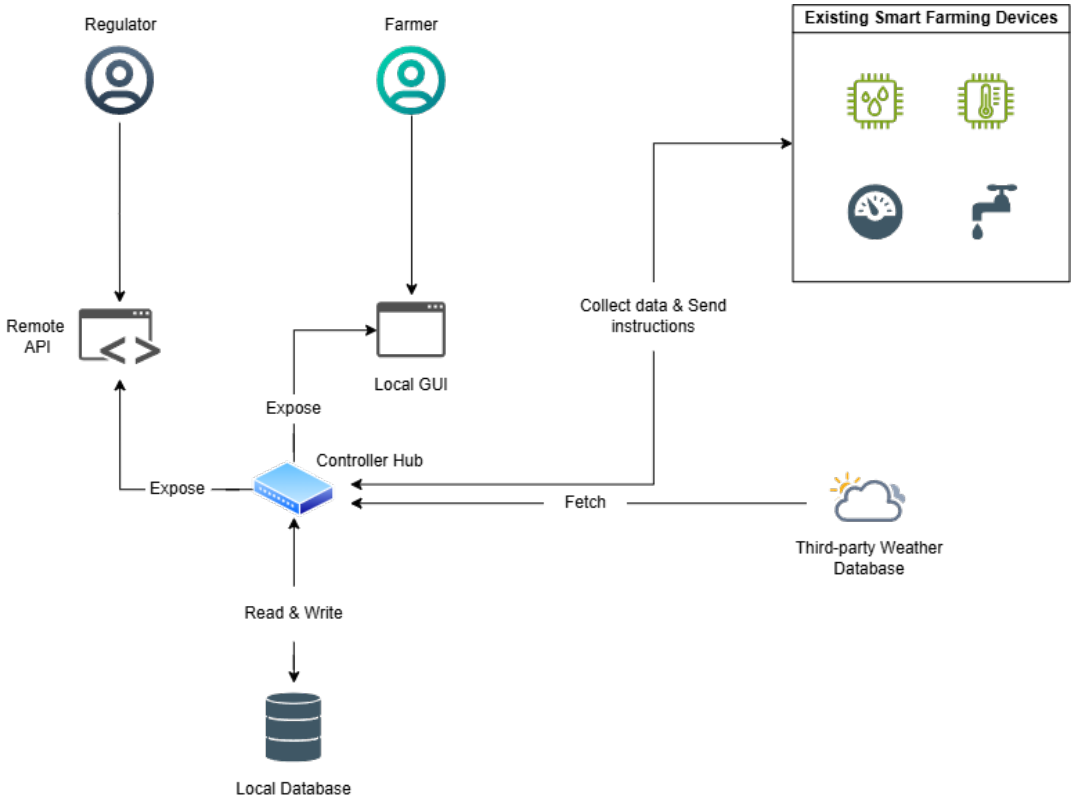


Figure 3.1: Architectural Vision diagram

3.2 Stakeholders and their Concerns

The smart irrigation system involves stakeholders with different interests.

- **Farm owners**
  Farm owners are our direct customers who pay to use the irrigation system. They are interested in reducing costs through automation and increasing long-term crop yields through more efficient use of resources. Their main concerns are the failure of the system, leading to loss of crops, and a poor return on investment. Commercial investors are a minor stakeholder that have very similar interests and concerns.
- **Farm workers**
  These are people interacting with the system on a day-to-day basis, and therefore want the system to be easy to work with. Their concerns are complications with the system, causing an increase in their workload, and unsafe working conditions.
- **Food safety regulators**
  Regulators represent and ensure the interests of the farm's consumers. They are interested in the production of healthy, nutritional food, that poses hygiene risk. Their concerns are noncompliance with safety regulations and product contamination.
- **Water regulatory bodies**
  These authorities are interested in maintaining safe water usage. Their main concern is water wastage.
- **Hardware suppliers**
  Farms purchase equipment from these suppliers. The system will work with farmer's

existing equipment or are supplemented with additional purchased hardware. This means that these suppliers have an interest in their hardware being compatible with our system. Their concern is our system not being compatible with their hardware causing sales to reduce.

- **Installers**
  People in charge of installing, maintaining and repairing the hardware and software integration locally are interested in being able to so with minimal effort. Their concerns are incompatible parts and unfixable failures.

- **Developers**
  Our developers are an important stakeholder as they are the ones in charge of maintaining the software and implementing new features as technologies develop. They are interested in a system that is easy to maintain and extend. Their main concern is poor system design and technical debt.

The stakeholders and their concerns are summarized in Table 3.1. Quality Attributes as defined in the ISO/IEC 25010 [1] standard are used to describe which characteristics are important based on the stakeholder concerns. ISO/IEC 25010 is part of the Systems and Software Quality Requirements and Evaluation (SQuaRE) family of standards. It provides a vocabulary for specifying and evaluating the quality of software products. The current standard defines nine quality characteristics, each of which is further divided into more specific sub-characteristics. The attributes relevant to the stakeholder concerns are defined as follows:

- **Performance efficiency** concerns the system's ability to perform within appropriate time and resource constraints
- **Compatibility** describes its ability to operate alongside other systems
- **Interaction capability** concerns how effectively users can interact with the system to complete tasks
- **Reliability** describes the system's ability to perform its intended functions consistently, including availability, fault tolerance, and recovery from failures
- **Maintainability** concerns how efficiently the system can be modified, repaired, and extended
- **Safety** describes the system's ability to avoid harm and danger to people, property or the environment

Stakeholder	Concerns	Quality attributes
Farm owners	Poor ROI	Performance efficiency
Farm workers	Compensatory work & risks	Interaction capability & Safety
Food safety regulators	Contamination	Safety & Reliability
Water regulatory bodies	Water waste	Performance efficiency
Hardware suppliers	Reduced demand	Compatibility
Installers	Catastrophic failures	Maintainability
Developers	Technical debt	Maintainability

Table 3.1: List of stakeholders

Some of the concerns listed are shared among multiple stakeholders, hence are more impor-

tant and need to be prioritized when taking decisions. These recurring concerns led to the definition of the system's key-drivers:

1. **Performance efficiency**: reduce the overall water consumption for farming activities
2. **Reliability**: ensure healthy crops for a safe and optimized production
3. **Compatibility**: allow various hardware from different manufacturers to interact with the system

3.3 Stories and Use cases

The system aims to provide a number of functionalities in order to address the concerns of stakeholders. Figures 3.2 and 3.3 offer a clear and intuitive visualization of these main functionalities through UML use case diagrams.

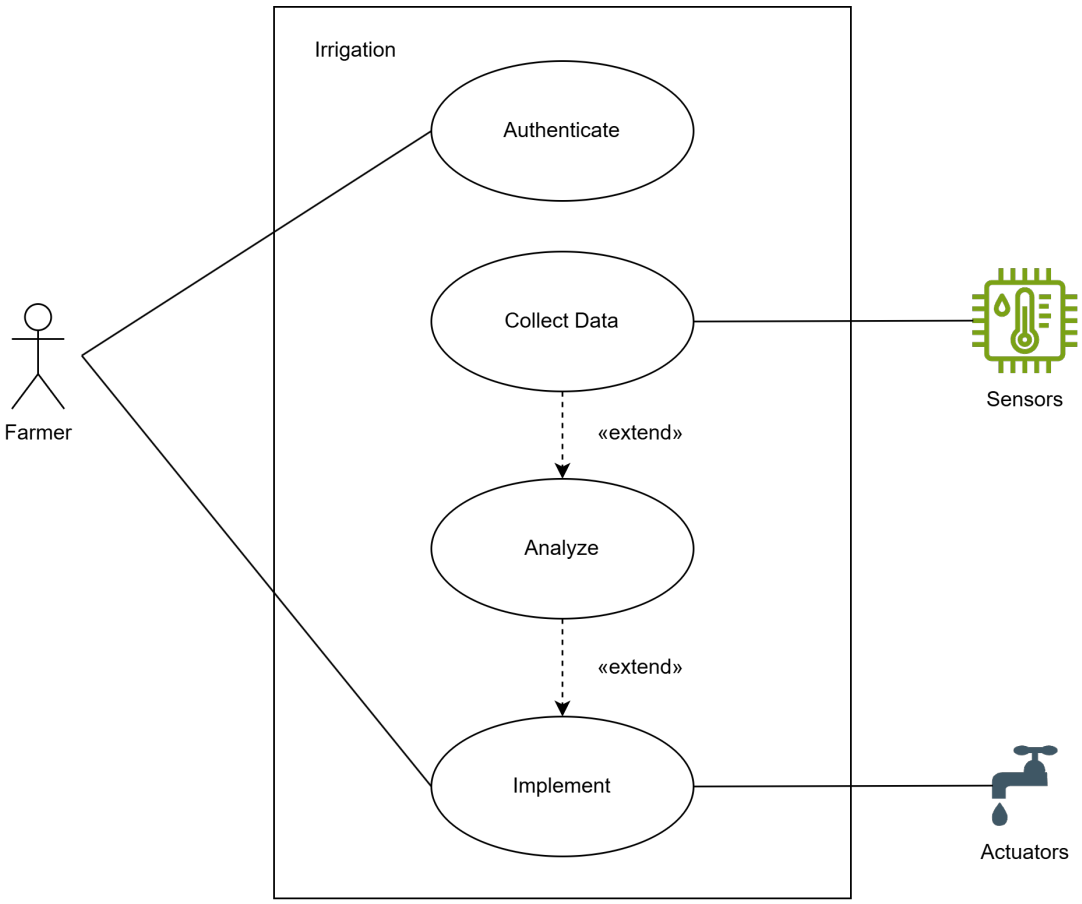


Figure 3.2: Irrigation Use Case Diagram

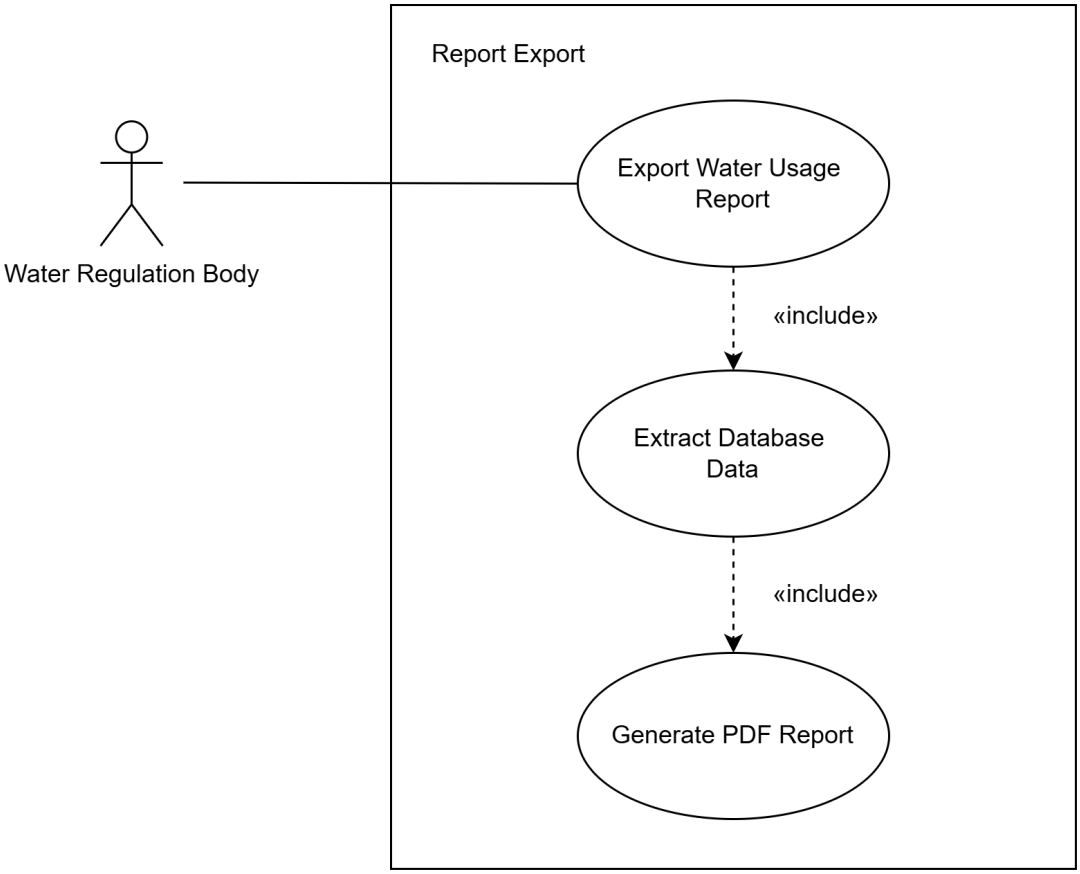


Figure 3.3: Report Export Use Case Diagram

UC1 - Collect data	
Primary actor	System
Goal	Collect soil data transmitted by up to 1000 sensors every 15 minutes, weather data every 6 hours, and manual user inputs
Preconditions	Soil sensors working and connected, third-party source reachable with an service uptime of 99%
Main success scenario	1. The central hub is listening for up to 1,000 requests/sec 2. The sensor sends 2 KB/packet of gathered data 3. The received data are saved in database under 100 ms
Extensions	1.a The central hub makes 3 data inquiry attempts to third party database (3 seconds connection timeout) 2.a The third-party provider sends the inquired data
Postcondition	Data is successfully saved to persistent storage with 0% data loss tolerance
Failing scenario	1. The central hub receives a burst of 1,000 requests in a single second 2. The system attempts to process and persist all 1,000 incoming packets (2 KB each) to the database concurrently 3. Congested communication tunnel or failing data link to database causes data loss or write operation slow down exceeding 100 ms
Related Requirements	FR-1, FR-2, FR-3

Table 3.2: UC1 - Collect data

UC2 - Analyze data	
Primary actor	System
Goal	Perform data analysis on collected data within 3 minutes
Preconditions	Successful data transfer from sensors to central hub - UC1
Main success scenario	1. Collected data are saved to the database under defined ingress rate 2. Data analysis component extracts and analyzes all recently saved data packet of 25-35 Bytes each under 3 minutes 3. Data analysis component sends irrigation decision to the next data processing component in the system chain
Extensions	
Postcondition	Returned irrigation decision is sent to the next component in the chain for further processing
Related Requirements	FR-4

Table 3.3: UC2 - Analyze data

UC3 - Track water use	
Primary actor	System
Goal	Track up to 100 concurrent implementing irrigation decision every 2 seconds and log total water volume consumed across all zones. This should happen nearly in real-time and allow verification that an irrigation decision has been correctly implemented
Preconditions	Water meters working and connected
Main success scenario	1. At least 25 water irrigation decision are in schedule 2. The water meter registers the total volume of water consumption with an accuracy of $\pm 1\%$ 3. The water use data is saved in the registry and added up
Extensions	
Postcondition	Operation water totals are permanently logged, updated on dashboards, and retained for 5 years for audit compliance
Related Requirements	FR-10

Table 3.4: UC3 - Track water use

UC4 - Export on-demand water use report	
Primary actor	Water regulatory bodies
Goal	Export an accurate up-to-date water use report on user demand
Preconditions	Water use tracking is working properly and accurately
Main success scenario	1. A water regulatory bodies member makes a report inquiry at a specific time logged as registered request time 2. The remote API backend negotiates with the orchestrator backend to access database without race conditions. 3. The remote API backend read database for latest data available at the registered request time 4. A PDF report of max 5 MB is generated and sent back to the requesting source within a total operation time of 10 to 30 seconds under typical Internet connection conditions.
Extensions	
Postcondition	Under typical Internet connection condition, within 10 to 30 seconds, a PDF file is transferred to the requesting source
Related Requirements	FR-10

Table 3.5: UC4 - Export on-demand water use report

UC5 - Implement irrigation decision	
Primary actor	Farmer
Goal	Implement automatic data-based or user-instructed irrigation decision
Preconditions	Functional authentication service & stable connectivity between controller hub and remote devices
Main success scenario	1. An irrigation decision is generated based on collected data or by an authenticated farmer using the dashboard 2. Within 5 seconds the concerned devices start implementing the irrigation decision
Extensions	
Postcondition	All irrigation decisions are being correctly implemented within 5 seconds
Related Requirements	FR-7

Table 3.6: UC5 - Implement irrigation decision

UC6 - Authenticate user	
Primary actor	Farmer
Goal	Authenticate an user to grant access to authorized actions
Preconditions	Running functional authentication service and database
Main success scenario	1. An user uses the dashboard to log in 2. Controller hub securely fetches credentials from database and performs checks 3. If the credentials are correct, the controller hub grant user access to authorized actions 4. The controller hub logs the number and time login attempt, the result of the authentication and the time of successful authentication
Extensions	
Postcondition	Only registered operators have access to the system under defined action authorization scope
Related Requirements	FR-5

Table 3.7: UC6 - Authenticate user

3.4 Functional Requirements

This section outlines the functional requirements for the Smart Irrigation system.

FR-1	**Must**	The system must asynchronously gather telemetry such as soil moisture, temperature, humidity, pH, and water levels from field sensors via MQTT.
FR-2	**Must**	The system must store all incoming sensor telemetry, historical logs, and user configurations in a persistent database.
FR-3	**Must**	The system must fetch and persist external provider data such as weather and evapotranspiration forecasts (via Weather APIs)
FR-4	**Must**	The system must process stored field telemetry and external weather using closed-loop (PID) control logic to make irrigation decisions.
FR-5	**Must**	The system must authenticate user credentials before granting access to dashboard, operational controls, or system configuration settings.
FR-6	**Should**	The system should monitor field gateway connectivity and automatically detect and log when a sensor node or the entire gateway becomes unresponsive.
FR-7	**Must**	The system must allow field operators to manually make irrigation decisions from the dashboard.
FR-8	**Should**	The system should flag anomalous telemetry readings (e.g., sudden drops in water pressure or extreme soil dryness) and alert operators prior to executing scheduled irrigation.
FR-9	**Should**	The Field Gateway should automatically synchronize all cached telemetry data with the Central Server once connection is re-established.
FR-10	**Should**	The system should export telemetry logs and generate automated water usage and compliance reports in PDF.
FR-11	**Must**	The system must give access to aggregated data and usage trends to farm operators through a dashboard interface.

3.5 Commercial Non-Functional Requirements

The following section describes the commercial non-functional requirements for the envisioned system:

CF-1	**Must**	The user interface of the system must be intuitive and easy to understand, such that users without technical backgrounds are able to use it efficiently. To this end, a system manual must be offered in the official language of each country the system is deployed to.
CF-2	**Must**	The price for running the system must not exceed 2.000 € for the entire software.
CF-3	**Must**	The system must offer users two distinct payment plans: the first must be an annual subscription, with updates and maintenance included in its price, while the second one must be one-time payments for major updates and maintenance billing as needed.
CF-4	**Must**	The total cost for system maintenance and updates must not exceed 50 € per hectare for the first year of the subscription plan. Price for the pay-as-you-go plan will be calculated based on the pending request plus a 20% profit margin.
CF-5	**Should**	In case of termination of service, the customer should be notified 6 months in advance and have the ability to export their entire usage history.

3.6 Technical Non-Functional Requirements

This chapter outlines the technical non-functional requirements, which are categorized in accordance with important quality attributes that were previously identified: reliability, performance, compatibility, maintainability, and security.

1. Reliability

TNF-1.1	**Must**	The system should be available 99% of the time. This is measured monthly and excludes maintenance periods.
TNF-1.2	**Must**	The system must resume operation within 5 minutes of recovering from an outage, without user input.
TNF-1.3	**Must**	In the event of a sensor connectivity failure, the system must not stop its activity, and instead use the last valid recorded measurements for that device for its next prediction.
TNF-1.4	**Should**	In the case of missing data when gathering sensor readings, the system should alert users within 10 minutes.
TNF-1.5	**Should**	Upon the installation of an update, down-time for the system should not exceed 10 minutes.
TNF-1.6	**Should**	The system should log all faults and failures with timestamps. These logs should be retrievable for a period of up to a year.

2. Performance efficiency

TNF-2.1	**Must**	The system must support concurrent data processing for at least 1000 sensors per farm without impacting response times.
TNF-2.2	**Must**	The system must generate predictions within 3 minutes of receiving all needed sensor and third-party data.
TNF-2.3	**Must**	The system should maintain a prediction accuracy within a 10% margin of actual optimal water requirements, validated through past outcomes.
TNF-2.4	**Must**	In the case of a new sensor addition, its installation, connection, and testing must not interfere with normal system operation.
TNF-2.5	**Must**	The user interface must load its components within 5 seconds after startup under normal network conditions.
TNF-2.6	**Should**	The system should generate and export usage reports within 1 minute under normal network conditions.

3. Compatibility

TNF-3.1	**Must**	At launch, the system must be compatible with sensors and irrigation devices produced by at least 3 major manufactures in the domain.
TNF-3.2	**Must**	The system must support standard IoT communication protocols for device integration. (e.g. MQTT, CoAP, HTTP).
TNF-3.3	**Should**	The system should be compatible with third-party weather services, following open and public API documentation.

4. Maintainability

TNF-4.1	**Should**	The system should allow developers to integrate new sensor types within 2 weeks of effort, without requiring a full redeployment.
TNF-4.2	**Should**	The system should support remote software updates without requiring on-site visits.

5. Security

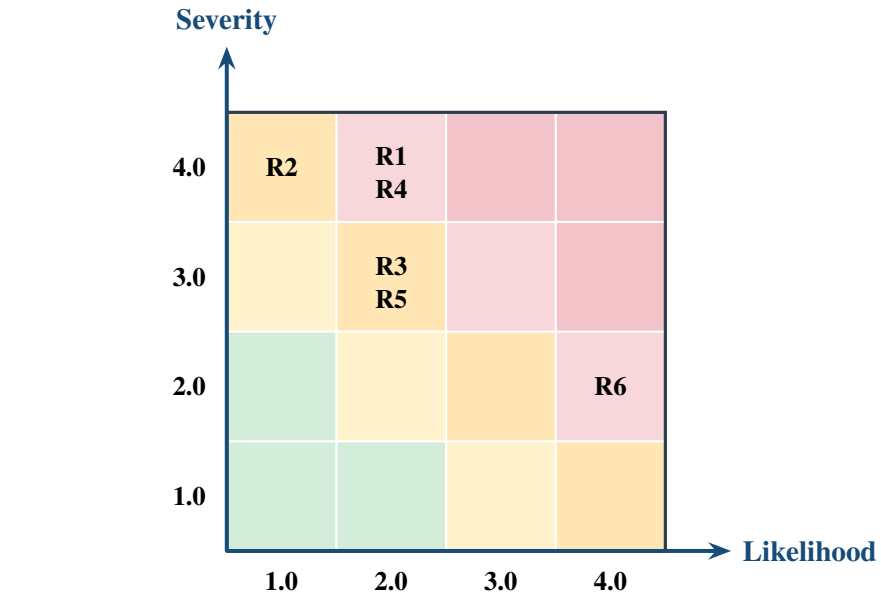
TNF-5.1	**Must**	The system must comply with GDPR requirements for storage and processing of farm and user data, given its deployment within the EU.
TNF-5.2	**Should**	All data transmitted between the system's backend, third-party gateways/APIs, and the user interface should be encrypted using TLS 1.2 or higher. Communication between individual sensors and their respective gateways should follow the encryption standard native to their communication protocol.
TNF-5.3	**Must**	To access system functionalities, roles must be distinguished between operators, owners and regulators.

3.7 Evolution Requirements

Agricultural technology and the European Mediterranean farming market are expected to change and evolve over time. The architecture should therefore accommodate foreseeable changes in supported hardware, crops, external services and its deployment environment.

ER-1	**Must**	The system must support future integrations of new types of hardware and equipment without requiring changes to the irrigation decision and automation functionalities.
ER-2	**Must**	The system must allow additional crop types to be introduced together with their irrigation parameters without redesigning the core system.
ER-3	**Should**	The system should allow existing weather providers to be replaced with new providers without affecting unrelated system functionality.
ER-4	**Should**	The core system should be able to function on farms with unreliable or non-existent internet connectivity. However, cloud-based storage and services should be supported when connectivity improves.

3.8 Risk Assessment



Low Risk | Moderate Risk | High Risk | Critical Risk | Very Critical Risk

		Likelihood	Severity	Level
R1	**Sensor data corruption** Sensor data got corrupted during transfer or due to hardware failure	2.0	4.0	**8.0**
R2	**Proprietary protocol lockdown** Devices use proprietary protocol and do not expose third-party integration compatibility	1.0	4.0	**4.0**
R3	**Product rejection due to complicated user interface** A non-streamlined UX/UI may cause user frustration and product rejection	2.0	3.0	**6.0**
R4	**Crop damage & yield loss due to improper PID tuning or flawed rule logic** Closed-loop control relies on deterministic rules and sensors; incorrect threshold configurations, sensor drift, or poor PID tuning may cause improper actuation and crop damage	2.0	4.0	**8.0**
R5	**Product liability scope in case of crop damage** The system makes use of automated closed-loop control and rule-based decision-making, raising questions regarding liability for design, sensor calibration, or tuning errors	2.0	3.0	**6.0**
R6	**Rural temporary Internet connection dropouts** Rural areas have limited cellular and/or wired network connectivity exposing the farm site to frequent temporary connection dropouts	4.0	2.0	**8.0**

Table 3.8: Risk Table

4 — Analysis

First, this chapter provides preliminary assumptions about the system and the context around it. After that, the technical roadmap which guides development is explored in detail. Lastly, high-level design decisions are described using ADRs.

4.1 Assumptions

The following assumptions are made about the context of the Smart Irrigation system:

- The Mediterranean region requires a precision smart irrigation system to optimize water consumption and limit waste.
- The farm already has a stable electrical connection and basic irrigation tools, such as pipes or pressure regulators.
- The farm owns or will purchase the necessary hardware to run the system.
- Third-party hardware, such as flow meters, sensors, etc., provides open standard protocols and/or APIs for seamless integration.
- End-users have access to internet connection and basic digital literacy.
- The central server will be deployed in a cool room without direct exposure to the sun or hot temperatures.

4.2 Technology Roadmap

Our team of software architects foresees four phases in order to develop the system. The first two phases focus on the main functionalities that the system is expected to offer, while the last two will provide significant upgrades in terms of reliability and safety.

The aim of the first phase is to realize a system capable of effectively collecting data about soil characteristics and variations, as well as crops health, and water consumption for both agricultural use and waste. This will be accomplished by setting up a pilot software connected to sensors and flow meters. The second phase focuses on adding a layer of automation to the system, making it possible to set irrigation and fertilization rules, in order to have an automatic decision making process to instruct the actuators.

The third phase targets reliability, providing cloud storage, in addition to the already deployed local database, that will act as a backup resource and will allow data to be shared among multiple locations of the same farm owner. In the fourth phase, access to consumption and production reports will be granted to supervision authorities, such as water or food regulators. These entities will be able to generate reports with the latest available data, to keep track of the farm's activities.

The foreseen technical roadmap is summarized in Table 4.1.

Table 4.1: Technology roadmap

Period	Main focus
Phase 1	Crops, soil, and water data collection
Phase 2	Autonomous irrigation
Phase 3	Cloud synchronization
Phase 4	Regulator reporting

4.3 High-level Design Decision

In this section, the major high-level design decisions are documented, together with their alternatives, advantages/disadvantages, and rationale. These decisions are described using Architecture Decision Records.

Table 4.2: ADR - Data Storage

Name	**Data Storage**
Decision	HDD-01
Status	*Approved*
Problem	The system needs to store the collected data from sensors in order to analyze soil metrics, plan irrigation efficiently, and report to end users.
Decision	Adopt a Local-First data storage model where all the collected data are written locally to an edge database and asynchronously synced to the cloud when an internet connection is available.
Alternatives	*Cloud-Only Database* • *Advantages:* Easy to scale, remote access and disaster recovery. • *Disadvantages:* High reliance on constant connectivity; complete loss of data logging during rural network dropouts. • *Feasibility:* Low. Dependent on stable internet, making it unreliable for remote fields with poor network coverage. *Local-Only Database* • *Advantages:* Full control over data and complete offline functionality. • *Disadvantages:* High physical risk, no off-site backups, no external API support, and hard to scale. • *Feasibility:* Low. Limits long-term data analysis, lacks remote monitoring, and cannot integrate external APIs.
Rationale	The local-first approach guarantees high fault tolerance against possible rural network outages while enabling cloud-based monitoring whenever connectivity is restored. On the other hand, cloud-only storage creates a single point of failure that compromises up-to-date irrigation monitoring in the case of network dropouts.

Table 4.3: ADR - External Weather Data Integration

Name	**External Weather Data Integration**
Decision	HDD-02
Status	*Approved*
Problem	The system requires weather forecasts and temperature trends to dynamically adjust irrigation schedules.
Decision	Integrate a Third-Party Weather REST API (Open-Meteo) via HTTPS to fetch hourly forecasts and reference ET_0 values.
Alternatives	*On-Site Weather Station* • *Advantages:* Completely self-contained, no external API dependencies, highest precision. • *Disadvantages:* High upfront equipment cost (anemometers, radiation sensors). • *Feasibility:* Low. Expensive solution and requires additional technical hassle.
Rationale	Third-party weather APIs provide high-accuracy predictive forecast data without requiring expensive local meteorological equipment on every farm.

Table 4.4: ADR - Security and Access Control

Name	**Security and Access Control**
Decision	HDD-03
Status	*Approved*
Problem	Telemetry data and valve control signals must be protected against eavesdropping, unauthorized access, and malicious command injections.
Decision	Implement Token-Based Authentication (JWT) for users and Lightweight Payload Encryption (AES-128) over MQTT channels for field sensors and valves.
Alternatives	*Unencrypted / Plaintext Communication* • *Advantages:* Low power consumption and zero processing overhead on sensor nodes. • *Disadvantages:* Vulnerable to packet sniffing, topic eavesdropping, and unauthorized valve actuation. • *Feasibility:* Low. High physical risk to farm infrastructure. *Heavy Enterprise Cryptography* • *Advantages:* Enterprise-grade security standard. • *Disadvantages:* Causes excessive memory consumption, high processing latency, and power drain on resource-constrained IoT nodes. • *Feasibility:* Low. Over-engineered for simple farm hardware.
Rationale	Lightweight encryption on MQTT messages secures field actions without overloading low-cost microcontrollers. Combining this with token access gives strong end-to-end protection with minimal resource usage. [29]

Table 4.5: ADR - Field Gateway Network Architecture

Name	**Field Gateway Network Architecture**
Decision	HDD-04
Status	*Approved*
Problem	Field hardware devices require continuous, low-latency communication with the central server.
Decision	Deploy a Field Gateway to collect local MQTT telemetry, route pulse signals to valves, and bridge communication with the central server.
Alternatives	*Direct Connection* • *Advantages:* Simplified network by eliminating an intermediate edge layer; removes the field gateway as a single point of failure in the field. • *Disadvantages:* Lack of edge rate-limiting exposes the central server to request flooding; high risk of server overload during reconnect cycles; inability to perform local data batching or offline queuing. • *Feasibility:* Low. High operational risks regarding central server scalability and high running costs.
Rationale	Centralizing local node traffic through an edge gateway provides rate limiting, payload aggregation and data compression, controlling data traffic to prevent server overload.

5 — System Architecture

5.1 Initial Model

In this section, a preliminary model of the system architecture is presented. Alongside the Smart Irrigation system that is to be developed, the model comprehends the Field Hardware and the Third-Party Weather Service. The former includes soil sensors to collect data and actuators to perform farming operations, while the latter provides weather forecasts and evapotranspiration data. The Cloud Backup maintains remote copies of both farming and users data. Two main external actors are represented in the diagram: regulators, such as food and water management supervision authorities, and farmers, symbolizing customers directly interacting with the system. The context diagram for the Smart Irrigation system is shown in Figure 5.1.

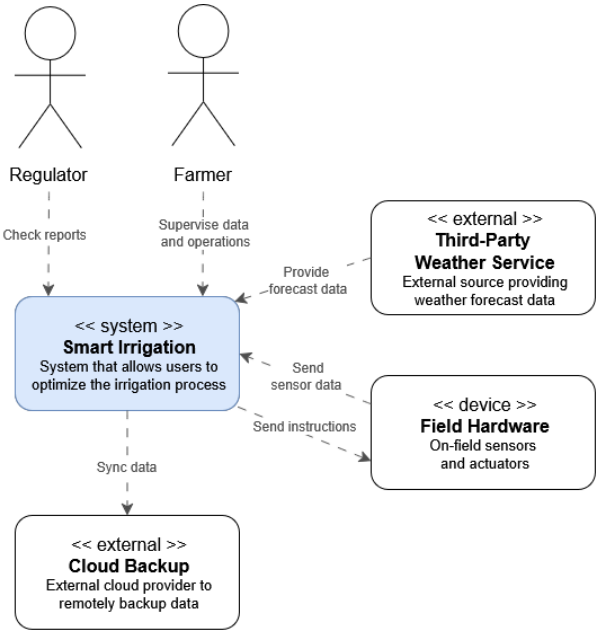


Figure 5.1: Context Diagram for the Smart Irrigation system

5.2 Elaborated Model

We hereby detail the main services that the system is composed of and how they interact with each other. The system is designed to provide fault tolerance and high availability, therefore it will not be developed as a monolithic architecture, but rather as a set of microservices, that will communicate through a client-server type of architecture. Figure 5.2 presents these microservices, highlighting how they connect to both internal and external components:

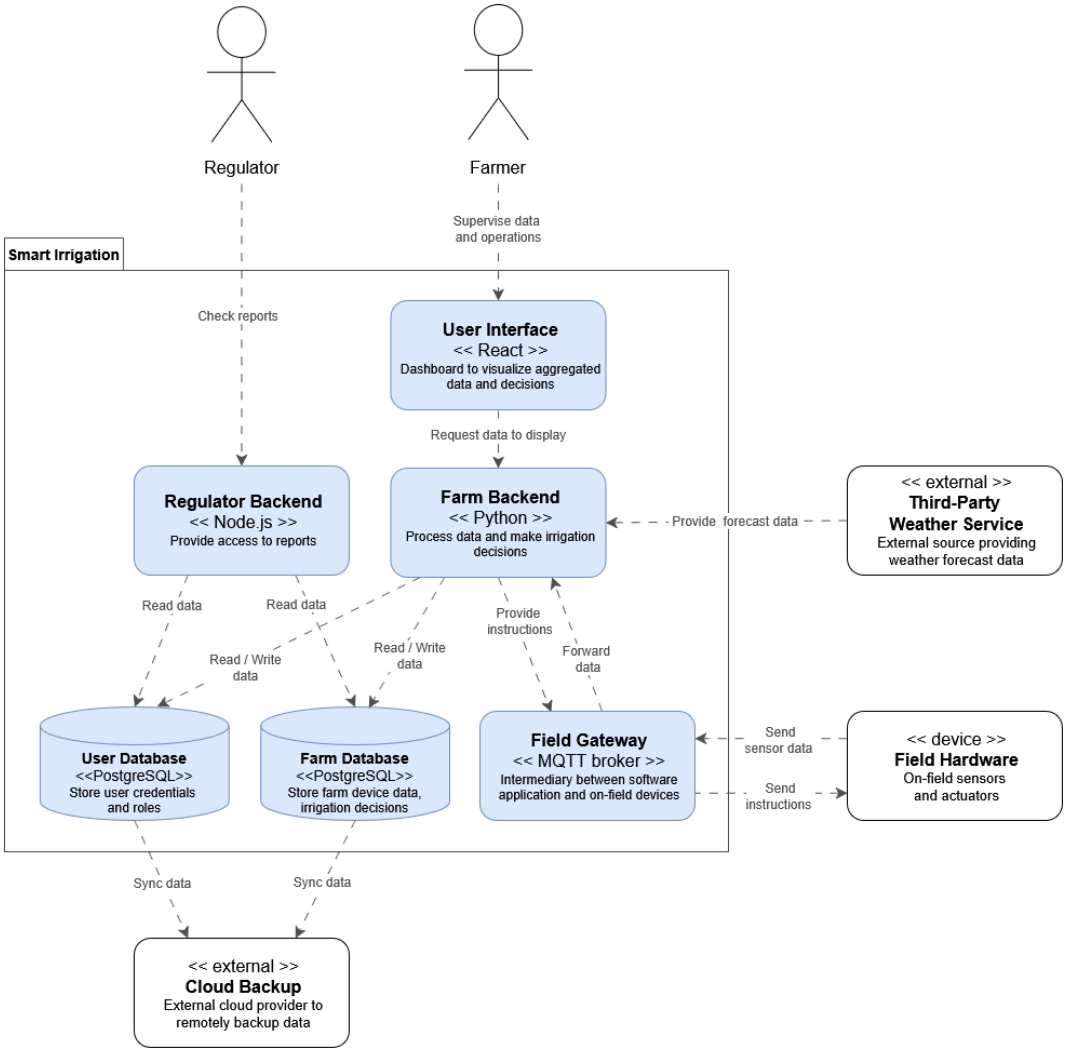


Figure 5.2: Service architecture diagram

- **UI**: an intuitive and easy to use graphical interface where, after successful authentication, the users are able to access dashboards in order to visualize aggregated data, trends and irrigation decisions, and if necessary, manually intervene in the operations flow.
- **Farm Backend**: the fundamental part of the system, whose main tasks are processing the data collected from the sensors and the external services and making irrigation decisions based on them. Not only these, the Farm Backend is also responsible for authenticating users and providing data to the UI.
- **Regulator Backend**: a separate backend service that handles the generation of production and consumes reports and provides authenticated access to them via API.
- **Farm Database**: a local database used to memorize soil data collected from the sensors as well as irrigation decisions made and water consumption data. It is periodically backed up to separate local storage and synchronized with the cloud.
- **User Database**: a local database that stores credentials and roles of users, including operators, owners, and regulators. It is backed up the same way as Farm Database.
- **Field gateway**: a lightweight message broker that acts as an intermediary between the

software application and the devices, providing protocol translation and flow control.

5.2.1 Technologies rationale

The system should rely on well established and widely used technologies in order to make use of public libraries and standards, as well as to ensure compatibility with different ecosystems. For this reason, the microservices that our system includes will be containerized using **Docker** [21]. These Docker containers will run inside a local **Kubernetes** cluster [14], to ensure fault tolerance and fast recovery. The need for reliable technologies extends also to the microservices themselves, in particular:

UI according to the premises, and considering the affinity of the team for the technology, the dashboard will be implemented using **React**, a popular JavaScript frontend library [24]. Other options included widely adopted JavaScript frameworks such as Vue [43] or Angular [17], but these ones would have required some overhead in order to get familiar with the features offered.

Farm Backend since the working backend will have to process possibly large amount of data, connect to the database, expose endpoints and communicate with the field gateway, a suitable option that can satisfy all these requirements is a backend developed in **Python** [16]. This choice is also compatible with a future implementation of the ML-based decision making process (Section 9.1.1).

Regulator Backend on the other side, the regulator backend only has to provide access to the data stored in the database, so the decision is to use **Node.js** [15] to supply a lightweight solution.

Farm Database in order to store data in a safe way, the choice fell on **PostgreSQL** [20]. Additionally, this database offers extensions to simplify the management of time series data (i.e. data points indexed by chronological order).

User Database considering its strong data integrity, support for ACID transactions, and structured schema, also the user credentials will be stored in a **PostgreSQL** database.

Field Gateway for field sensors and gateways communication, MQTT is the de facto standard [26], hence the technology chosen is a message broker based on this protocol, that is **Mosquitto** [3]. An alternative to this choice is Kafka [13], but it introduces complexity to a system that does not need sophisticated functionalities.

As for the remote backups, they will be hosted on an external cloud provider infrastructure. In order to maintain high reliability on this side, the choice has fallen on **Amazon Web Services** [4], that is the dominant global cloud market share leader as of second quarter 2026 [38], able to offer 99.99% availability and 99.999999999% durability [5]. Other viable options could have been Google Cloud [18] or Microsoft Azure [25].

5.3 Alternative models

5.3.1 Cloud-based service

An entirely different approach for the development and deployment of the system has been explored as an alternative to the on-site model. The fundamental point of this second option is to host the Smart Irrigation system remotely, serving it to customers as a cloud-based service. The technology stack chosen for the development of the software application would have been the same. However, the main difference would have resided in the separation of

concerns between on-site components for farming operations (the Field Gateway) and remote components (Farm Backend, User Interface, etc.).

The major advantage of this kind of solution is having a centralized application, scaled as needed, that can be served to multiple customers seamlessly. Additionally, the farm would benefit from a reduced upfront cost for hardware, deploying only the resources necessary to interact with the remote system. Other advantages on the provider side are easier maintenance and higher profit margins. Having said that, the major drawback, and reason not to proceed with this model, is the exigency of a continuous strong internet connectivity, which most farms are not provisioned with. Many types of crops need daily care to achieve optimal growth, so it would not be acceptable to completely idle the irrigation system following a prolonged connection outage. On the hardware costs side, some devices would still be necessary to exchange information with the remote software and to store a local backup of the data.

5.3.2 Direct field communication

An alternative architecture was also considered, consisting of a database for data storage, one backend dedicated to farming operations, a second backend exposing an endpoint for the supervision authorities, and the UI to easily visualize data and trends. This design dropped the field gateway as an intermediary, assuming instead a direct communication between the system and the sensors over a parallel connection.

The benefits of this solution are the absence of a single point of failure and the possibility of scaling connections by running multiple instances of the farming backend in parallel. The drawbacks, however, lie mostly in implementation: handling the sensor connections and keeping data consistent across them would be considerably harder, and this could affect the reliability and efficiency of the system.

5.3.3 In loco weather sensors

Another approach for weather and evapotranspiration data collection is to deploy weather sensors on site, or, since some farmers already have them installed, use the existent ones, and connect these to the rest of the system, precisely to the farm backend.

The main advantage of this solution is to get the most accurate data possible geographically speaking, not having to rely on data coming from cells arranged hundreds or even thousands of meters away. However, taking this road would lead to not negligible complications, such as an increase of upfront costs and hardware management concerns. To make matters worse, the majority of farmers do not have these kinds of sensor in operation, so resistance to new implementations should also be taken into account.

5.4 Verification

In this section, the feasibility of the system architecture is discussed. First the performance and capacity requirements are discussed and verified. Then, the reliability and availability are approximated.

5.4.1 Performance

In Use Case UC1 given in Table 3.2 we assume a maximum deployment of 1000 sensors where each sensor submits approximately 2 KB of data every 15 minutes. under operation at this upper limit we have an incoming message rate of 1.11 messages/s and an average

data rate of 2.22 KB/s. This average throughput is small. UC1 also specifies that the system should be able to handle up to 1000 simultaneous requests per second. Assuming the same 2 KB per request the corresponding peak bandwidth is $1000 * 2$ KB $= 2$ MB/s. Since we have a field gateway collecting the sensor data we can reduce the load placed directly on the backend by aggregating and limiting simultaneous throughput. This means that the average and peak data do not indicate network bottlenecks.

5.4.2 Capacity

Storage capacity is more significant. At the maximum UC1 scale, storing 2 KB measurement data from 1000 sensors every 15 minutes produces $1000 * 96 * 2$ KB $= 192$ MB per day. This is about 70 GB of raw data per year, not including logs, backups or irrigation decision data. This is a considerable amount of storage capacity is still feasible considering this is the upper limit of 1000 sensors. Farms with lower sensor counts can expect to store 10 GB or less per year. A retention and aggregation strategy is required for large deployments on large farms. Older measurements could, for example, be sampled into hourly or daily aggregates or selectively preserved.

5.4.3 Reliability and Availability

We require the system to achieve 99% monthly availability. Over a 30-day month this permits approximately 7.2 hours of total downtime. System availability is dependent on the availability of the hardware and software components included.

Availability can be estimated using Mean Time Between Failures (MTBF) and Mean Time To Repair (MTTR). We have

$$A = \frac{MTBF}{MTBF + MTTR}$$

where A is the availability. We first estimate preliminary MTBF and MTTR values for the likely hardware components that will be used:

[Insert component availability table]

Some components being unavailable will directly cause the system to be unavailable as well. Therefore it is important that the aggregate availability of these critical components stays above the specified 99% availability requirement.

Sensor failures do not prevent irrigation decisions to be made since the system is allowed to use the last valid reading when an individual sensor becomes unavailable. This practice improves fault tolerance, but failing sensors can still lead to unreliable data impacting irrigation decisions. Therefore, the existing requirement to notify users of missing measurements should prevent this from happening and increase reliability.

The field gate, database and system backend all function locally. This means that no internet connection is required to gather data, make irrigation decisions and execute these decisions. Systems that are connected to a cloud database can be resynchronized once connection returns.

6 — Hardware Architecture

6.1 Hardware Decisions

ID	Name	Description
HD-01	Controller Architecture	Deploy an ARM-based Central Processing Controller equipped with local processing capabilities, direct API integration, Wi-Fi, and scalable Ethernet connectivity to accommodate an expanding network of hardware Field Data Relay connections.
HD-02	Soil Moisture Sensor	Integrate a Volumetric Water Content (VWC) Capacitance Sensor (Truebner SMT100) to monitor soil moisture levels across fields.
HD-03	Flow meter	Given the existing climate concerns, the best fit for our system would be the Seametrics AG3000 Series Flanged Magmeter.
HD-04	Solenoid Valve	Given the existing climate concerns, the best fit for our system would be the Bermad IR-410-3W-RX.
HD-05	Connectivity	Adopt a Hybrid Network Architecture utilizing a low-power Zigbee mesh for local sensor-to-gateway telemetry and Wi-Fi for gateway-to-cloud communication.
HD-06	Power Supply	The system must deliver continuous, reliable electrical power to various field sensors and actuators (solenoid valves and pumps).
HD-07	Water pump	Given the existing climate concerns, the best fit for our system would be a Submersible Multistage Pump.

Table 6.1: A brief overview of the hardware decisions.

6.2 Hardware Overview

Smart Irrigation uses a modular and scalable hardware architecture composed of:

- A Central Processing Controller running Farm Backend software components
- An on-site Database

- A scalable number of Field Data Relays running a bidirectional software communication interface (Field Gateway) and working in pairs for fault tolerance
- A scalable number of third-party sensors and actuators

Smart LoRa-enabled sensors and actuators transfer data wirelessly to deployed Field Data Relays. Meanwhile, traditional industrial farming devices connect to existing third-party LoRa bridge interfaces to enable wireless communication with these relays.

Each Field Data Relay pair links to the Central Processing Controller via a wired Power over Ethernet (PoE) connection. Data is transmitted over shielded S/FTP CAT6A Ethernet cables, ensuring adequate bandwidth and interference-free, bidirectional long-distance transmission in environments crowded with Wi-Fi and LoRa signals.

If one Field Data Relay in a pair goes out of service due to a hardware or software failure, the remaining relay seamlessly steps in to ensure uninterrupted operation.

The Central Processing Controller connects to the on-site database using either high-speed copper or optical links, depending on the scale of deployed relays, sensors, and actuators. Additionally, the controller uses Wi-Fi to link to an Internet Service Provider (ISP) for third-party API integration and cloud database backups.

The entire architecture is powered by a redundant hybrid system that seamlessly combines solar and grid energy.

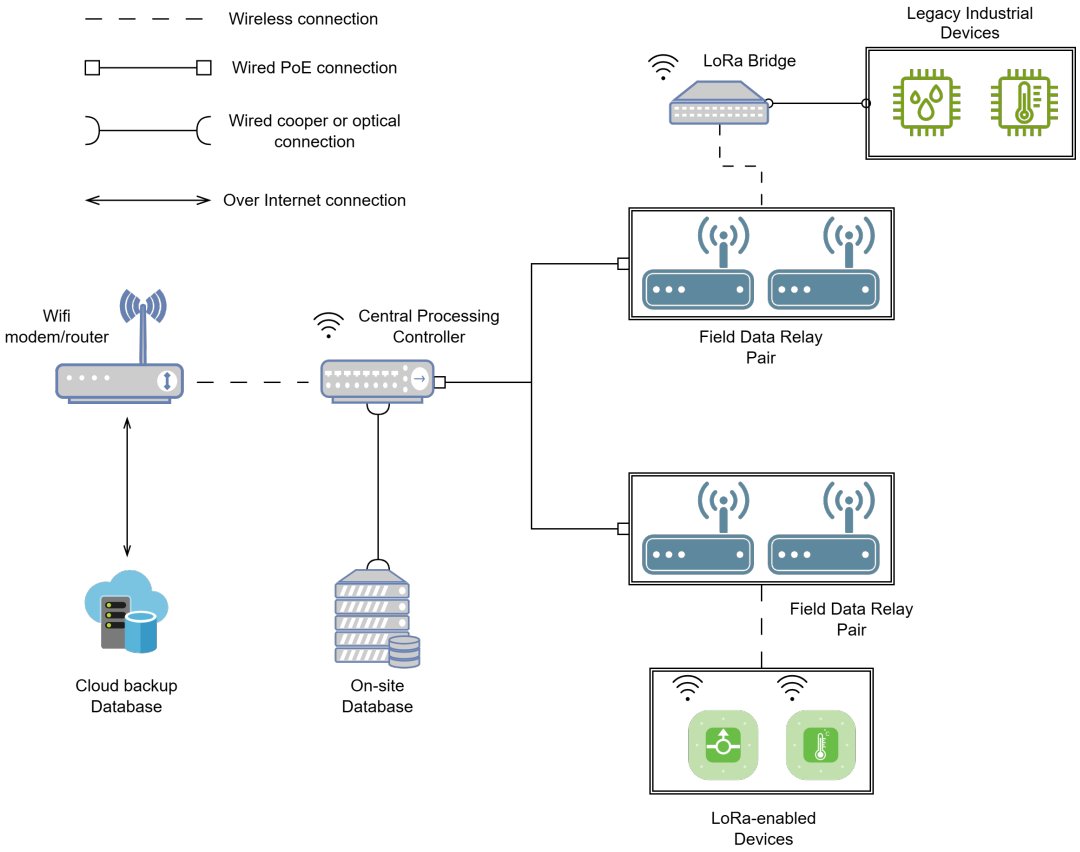


Figure 6.1: Hardware Overview

7 — Software Architecture

In this chapter, the software architecture of the system will be detailed. Firstly, an overview of the architecturally relevant drivers is provided. In the following sections, a selection of architectural views, based on the 4+1 view model [23], is presented. Finally, the software design decisions are described.

7.1 Architecturally Relevant Drivers

The following table contains a list of all architecturally significant requirements. These selected requirements closely relate to the key drivers and use cases we defined in prior sections, and influence the system's design and software architecture.

Type	Requirement
FR	FR-1, FR-2, FR-3, FR-4, FR-5, FR-6, FR-8, FR-9, FR-10
TNF	TNF-1.1, TNF-1.2, TNF-1.3, TNF-1.4 TNF-2.1, TNF-2.2, TNF-2.3,TNF-3.1, TNF-3.2
ER	ER-1, ER-2, ER-3, ER-4

7.2 Logical View

This section contains the logical organization of the services included in the system, as well as the allocation of functionalities among them.

7.2.1 Farm Backend Components

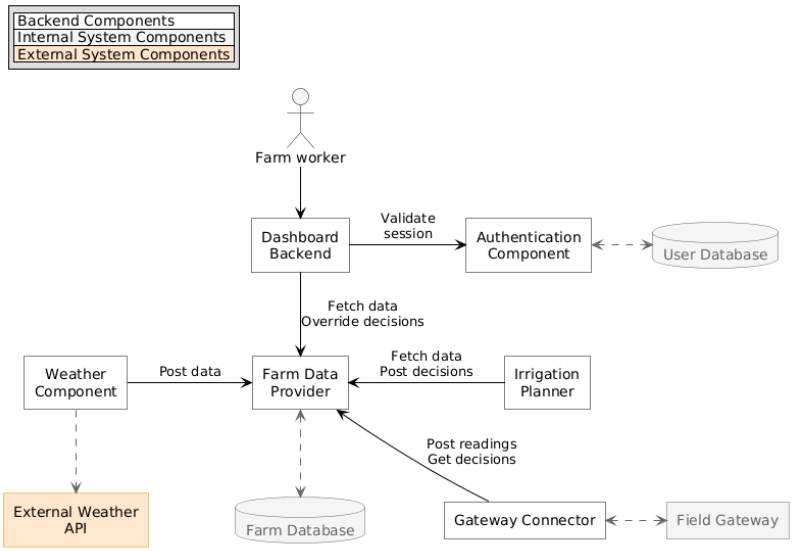


Figure 7.1: Farm Backend's Logical View diagram

Farm Data Provider The Farm Data Provider bridges communication between the farm database and other backend components, as all reading and writing of farm data is done through it. It stores sensor and water meter readings, weather forecasts obtained from external APIs, irrigation decisions, and user overrides.

Dashboard Backend The Dashboard Backend handles requests made by the frontend, fetching water usage and logs of past irrigation decisions to compute statistics. This component additionally allows users to override irrigation decisions: the override is posted to the Farm Data Provider, taking priority over automated irrigation commands.

Authentication Component The Authentication Component is used to manage user access to the system. It verifies user credentials, creating and validating user sessions for the Dashboard Backend.

Gateway Connector The Gateway Connector acts as the intermediary between the field gateway and the backend logic. It receives sensor and water meter readings. Pending irrigation commands (meaning commands that have not expired or been overridden by the user) are retrieved periodically and forwarded to the field gateway.

Irrigation Planner The Irrigation Planner computes precise water amounts for each irrigation zone, taking into account sensor and water meter readings, weather data, and past irrigation decisions. It then issues irrigation commands which are forwarded to the Farm Data Provider. This component does not communicate with the Gateway Connector, as all logic is routed through the farm database.

Weather Component The Weather Component fetches weather forecasts from a third-party API at predefined intervals and forwards them to the Farm Data Provider for storage.

7.2.2 Regulator Backend Components

Farm Database Reader The Farm Database Reader handles the interaction with the Farm Database, providing reading access to crops and water usage data registered.

Reports Generator The Report Generator is the component responsible for the aggregation of data and their presentation in a well-structured, human-readable PDF document. Additionally, other data formats are taken into account in this component, such as CSV and JSON, for more specialized and technical applications that the supervision authorities may exploit.

Regulator Server The Regulator Server exposes various API endpoints in order to let the supervision authorities retrieve the reports and the telemetry data generated by the system.

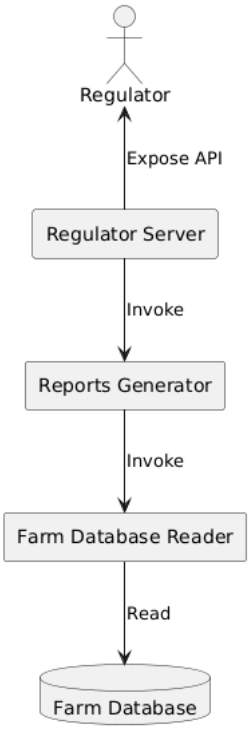


Figure 7.2: Regulator Backend's Logical View

7.2.3 Local Database Components

Figure 7.3 shows the schema of the local database.

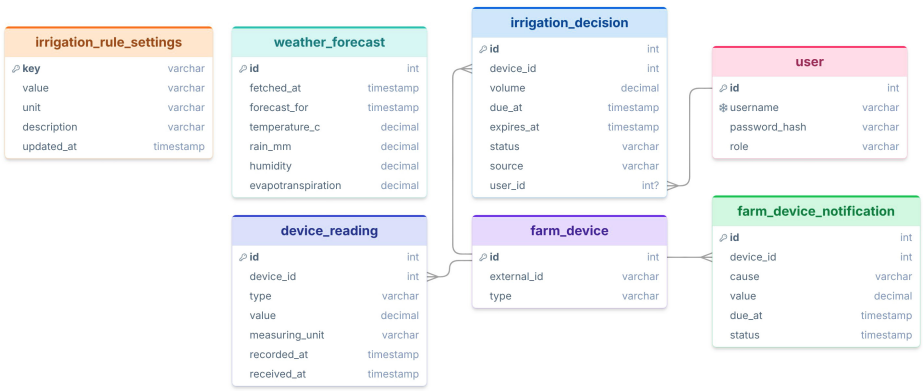


Figure 7.3: Diagram of the local database schema

Farm Device keeps a record of all on-field devices, such as sensors, flow meters, and valves.

Device Reading is used to store values reported by the farm devices (soil characteristics, water volumes, etc.). Each row holds the reading type, along with the corresponding measuring unit. Timestamps for when the reading was picked up and received are also included.

Weather Forecast holds forecasts fetched by the Weather Component, with the expected temperature, rainfall, humidity, and evapotranspiration for that date.

Irrigation Decision defines the irrigation instructions for a valve: a specific water amount, the time at which it is due, and the time after which it expires. Decision status and source are also included, indicating the author of the irrigation decision (irrigation planner or a user) and whether it was implemented, pending, or canceled.

Farm Device Notification is used for notifications that are raised in case of critical events, such as low soil moisture, valve failure, and low pipe pressure. It stores the cause, device source, value which triggered the alert, time, and the status of the notification.

Irrigation Rule Settings stores configurable parameters for the decision rules, defined as key-value pairs with a unit and description (for example rainfall thresholds or the water allowance).

User holds information regarding accounts that are authenticated to access the system. It stores the unique username, password, and the role of the user (operators, owners, and regulators). Passwords are never stored in plain text.

7.2.4 User Interface Components

Login Component The Login Component handles user authentication, client-server validation, and routes users to specific dashboards based on their assigned access roles.

Owner Dashboard The Owner Dashboard interface designed for farm owners displays aggregated metrics across all fields, overall water consumption, system health, hardware devices status.

Worker Dashboard The Worker Dashboard interface designed for field workers displays active irrigation schedules, assigned field tasks and quick-access valve controls.

Manual Operations The Manual Operations component enables authorized operators to manually trigger, pause or adjust durations of solenoid valves and water pumps on demand.

Notification Component The Notification Component displays real-time alerts of critical events (eg. low soil moisture, valve failure, low pipe pressure) via push banners and a dedicated notifications page.

Map of Devices The Map of Devices component is a interactive GIS grid that displays deployment locations and real-time operational statuses of field sensors, pumps and gateways.

Historical Data The Historical Data component renders analytical charts (soil moisture trends, temperature, rainfall, and water volume consumed) with customizable date ranges and export capabilities.

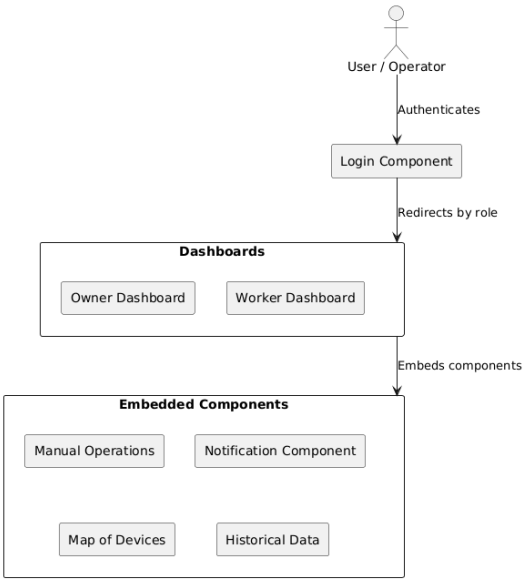


Figure 7.4: User Interface's Logical View

7.2.5 Field Gateway Components

Queue Manager (Device End) The Device-End Queue Manager handles incoming message queues from the field sensors. It acts as a buffer to temporarily store these messages if there is a network connectivity issue, preventing data loss.

MQTT-HTTP REST Translator The MQTT-HTTP REST Translator converts messaging formats and bridges protocols between MQTT (used by sensors) and RESTful endpoints (used by servers and cloud).

Queue Manager (Backend End) The Backend-End Queue Manager buffers and queues outgoing payloads to the central server or cloud.

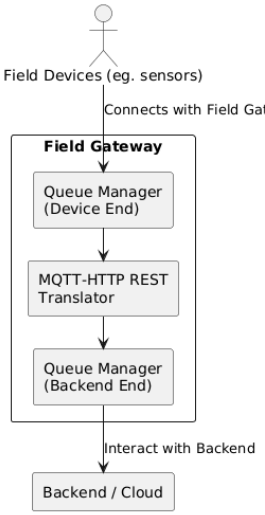


Figure 7.5: Field Gateway Logical View

7.3 Implementation View

This section describes the proposed Smart Irrigation system from a development perspective, showing how components from the Logical View are organized in the source code.

7.3.1 Repository Structure

The source code of the project is kept in a single repository. System services are located in their own directory within the main **smart_irrigation_system** folder. Service directories are additionally split into components which map directly to those introduced in the Logical view, as annotated in Figure 7.7. The top-level structure is as follows:

```
smart_irrigation_system/
├── UI/ ..................................................... React dashboard
│   ├── pages/
│   ├── features/
│   ├── shared/
│   └── api/
├── farm_backend/ ..................................... Python service
│   ├── dashboard_api/
│   ├── auth/
│   ├── irrigation_planner/
│   ├── weather/
│   ├── gateway_connector/
│   └── data/
├── regulator_backend/ ..................................... Node.js service
│   ├── server/
│   ├── report_generator/
│   └── data/
├── field_gateway/ ..................................... Queue managers and translator
│   ├── device_queue/
│   ├── translator/
│   └── backend_queue/ ..................................... PostgreSQL schema
├── contracts/ ..................................... OpenAPI, MQTT schemas
├── db/migrations/
└── deploy/ ..................................... Dockerfiles, Kubernetes manifests
```

Figure 7.6: Directory structure of system

The **deploy** directory contains no application code; it holds the files needed to deploy the system on a Kubernetes cluster according to the Deployment View.

7.3.2 Package diagram

Figure 7.7 provides a more in-depth view of the system modules and their dependencies. Solid arrows are build-time dependencies, while dashed ones show runtime communication between modules; external services are omitted.

Coupling between modules is overall low: the modules share no code, and the only artifacts they have in common are the **contracts/** (to keep their interfaces compatible), and the database schema in **db/migrations/**. Out of the two, the schema is a more significant dependency, since a change to a table can affect every module that queries it. However, this risk is limited due to the fact **regulator_backend** has read-only access to the farm data, and that the schema changes go through shared migrations.

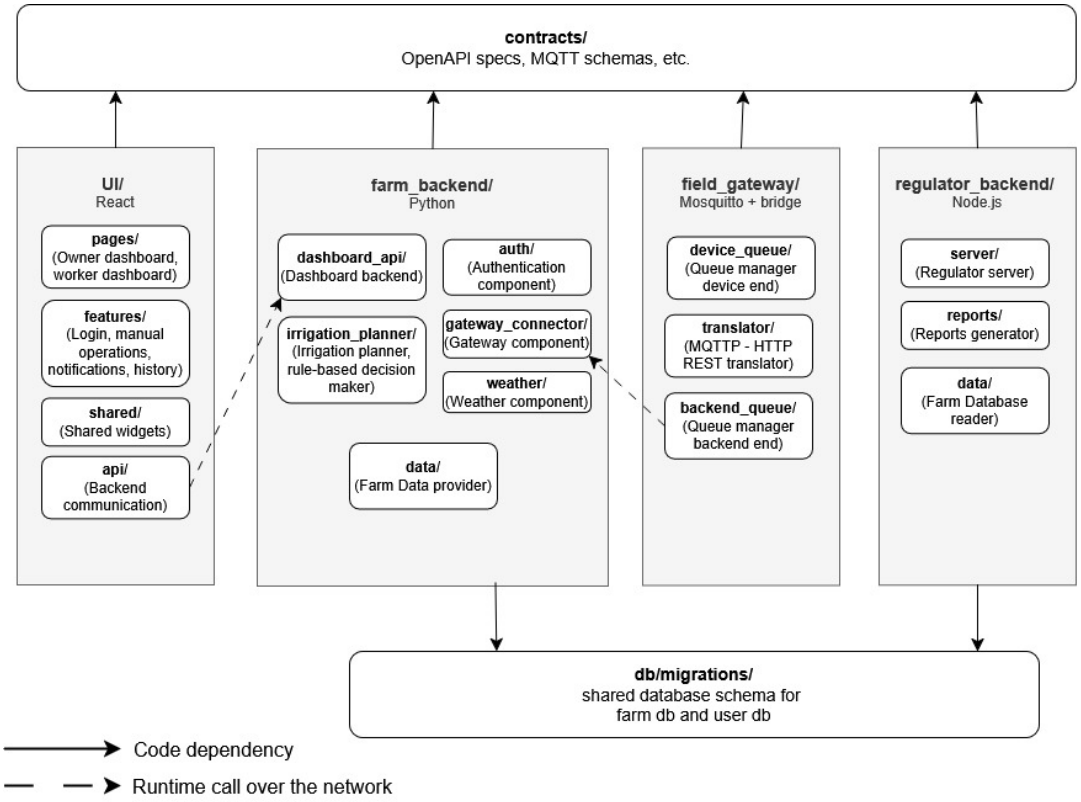


Figure 7.7: Package diagram of the system modules and shared artifacts

7.4 Process View

In this section, the processes and objects involved in runtime system operations are detailed, mainly focusing on the interactions between components needed in order to carry out the implemented functionalities. Two main scenarios are examined, with the purpose of depicting the operations and communication, for both the system as a whole and the Farm Backend, arranged in time sequence.

7.4.1 System Operations

Figure 7.8 represents the desirable operative scenario regarding the deployed system. To describe the procedure in chronological order, the first thing that happens is the Soil Sensor measuring some characteristics about the farm land. These data are pushed against the Field Gateway, which translates them in a readable format for the rest of the software, and then forwards this elaborated version to the Farm Backend. First, the Farm Backend updates the dashboard inside the User Interface with the new measurements. Subsequently, it starts processing the data in order to reach an efficient irrigation decision, that will be saved in the Farm Database, alongside the values it is based on, and displayed on the User Interface. Finally, the irrigation decision is sent to the Field Gateway, which forwards it to the Irrigation Valve for the action to be performed.

In the considered failure scenario, the Farm Backend does not receive any data from a specific Soil Sensor for five consecutive minutes. In this case, the Farm Backend emits a warning, displayed on the User Interface, and proceeds to retrieve the last measurement obtained by the aforementioned sensor, allowing the irrigation procedures to continue while the sensor is repaired or substituted.

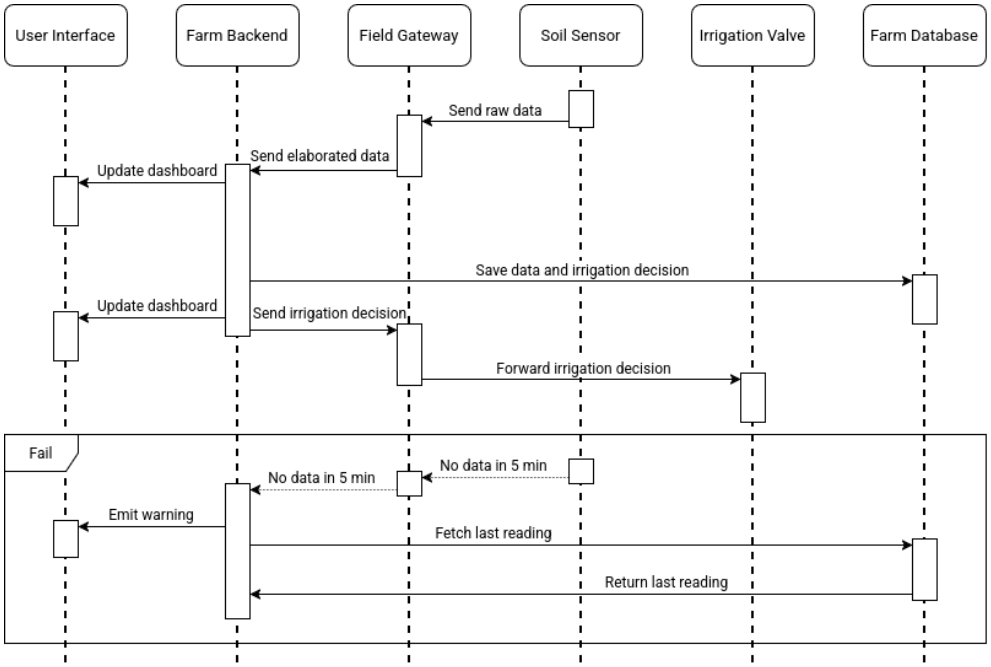


Figure 7.8: Sequence diagram - System Operations

7.4.2 Farm Backend Operations

Figure 7.9 shows the sequence of operations followed by the main components of the Farm Backend (white boxes), as well as the microservices strictly related to them (gray boxes), in an ideal operating scenario for the system. The start is given by the Field Gateway sending the data measured by the sensors in the farm to the Gateway Connector. From here, the data are forwarded to the Farm Data Provider, which publishes them on the Dashboard Backend as soon as they are received. Next, the Farm Data Provider fetches recent historical data from the Farm Database in order to have a larger dataset to compute the irrigation decision. This step is performed by the Irrigation Planner, which receives the data and returns the irrigation decision. At this point, the Farm Data Provider saves both the data and the irrigation decision in the Farm Database, publishes the decision on the Dashboard Backend, and sends it to the Gateway Connector. The last step is the Gateway Connector sending the decision to the Field Gateway, which will send it to the actuators.

Considering the previously mentioned failure scenario more in detail, it is now the Gateway Connector that notices the data absence for five consecutive minutes. In this case, the Gateway Connector emits a warning to the Farm Data Provider, which publishes it on the Dashboard Backend, for it to be displayed on the User Interface. Next, the Farm Data Provider fetches the last reading made by the missing sensor from the Farm Database and provides the data to the Irrigation Planner, ensuring the correct execution of the irrigation procedure.

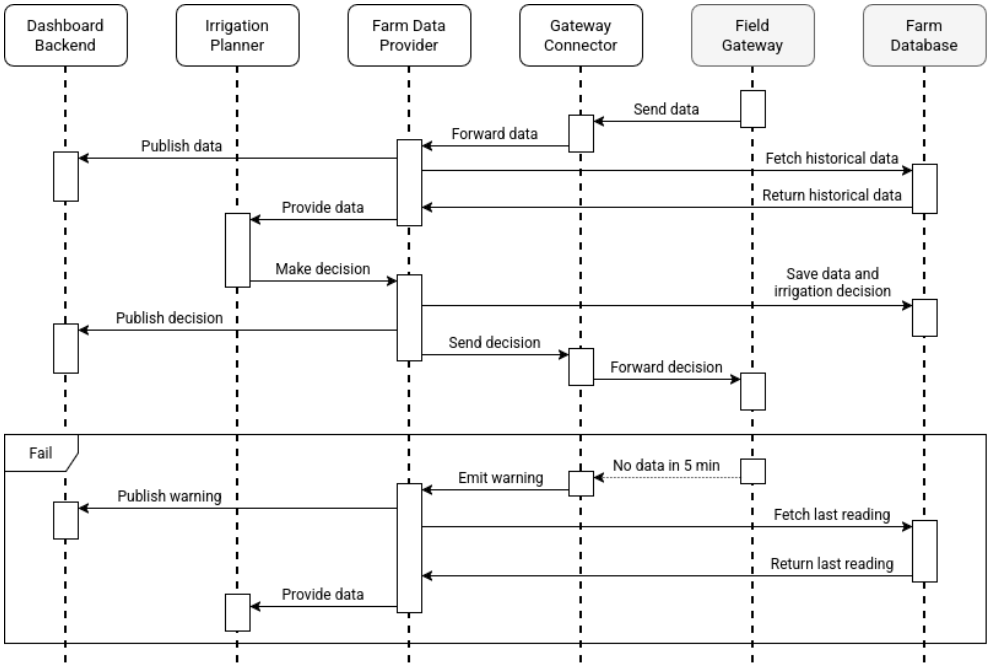


Figure 7.9: Sequence diagram - Farm Backend Operations

7.5 Deployment View

This section describes how the Smart Irrigation system will be deployed. Figure 7.10, which represents the physical distribution of the software, shows three main locations for the system: Cloud Storage, Local Kubernetes Cluster and physical devices on site, namely Field Data Relays, sensors and actuators.

7.5.1 Cloud Storage

On the Cloud Storage will be deployed two remote database backups, one for the Farm Database and one for the User Database, allowing the system to maintain high fault tolerance against physical threats. Currently, these two databases do not actively participate in the system's operations: their role is to store up-to-date information about the farm and its users, and to grant access to them when necessary.

7.5.2 Local Kubernetes Cluster

The location where the Kubernetes Cluster will be deployed is the Central Processing Controller. Here, the Local Kubernetes Cluster will host the microservices architecture of the system, including deployments for the Farm Backend, the Regulator Backend and the User Interface, keeping two pod replicas active for each instance of every deployment, providing availability, load balancing, and again, fault tolerance. Instances of the two databases will also be hosted within the cluster, in the form of StatefulSets.

7.5.3 Physical Devices

The last location in which the system will operate is the actual farm land, where the Field Data Relays, the sensors and the actuators will be deployed. To be precise, the sensors and actuators will not run any specialized software but the one they are packaged with, and they will interact with the system using common interfaces. For this reason, the Field Data Relays

will run the Field Gateway container, acting as an intermediary between the on-site smart hardware and the Central Processing Controller. The Field Data Relay is not to be considered a single point of failure; in fact, several physical devices can be deployed in different areas of the farm land, each of them connected to the Central Processing Controller, in order to split the workload and provide higher fault tolerance against hardware failures.

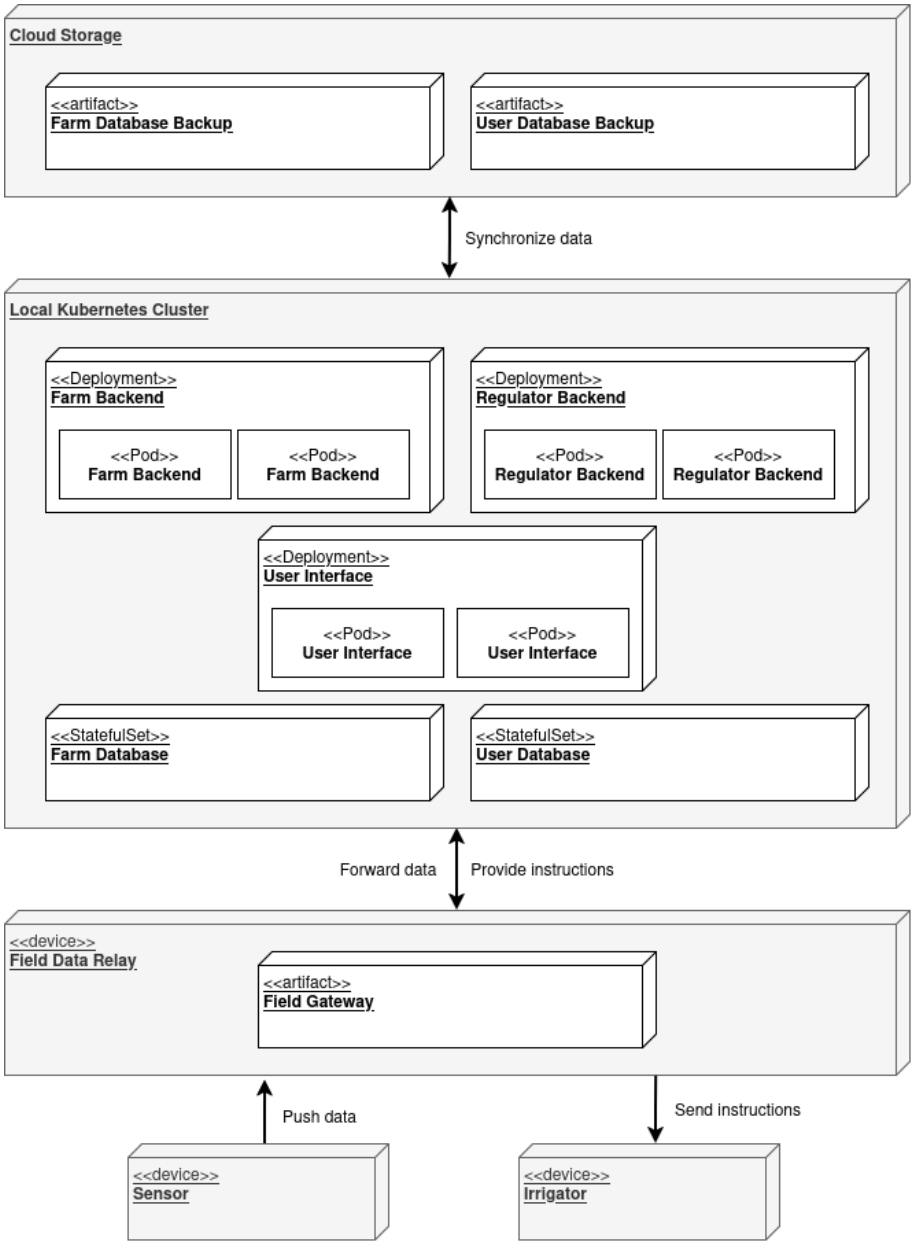


Figure 7.10: Deployment diagram

7.6 Software Design Decisions

Table 7.1: Decision - Architectural Style

Name	**Architectural Style**
Decision	SD-01

Status	*To review*
Problem	The smart irrigation system handles diverse functionalities which must scale independently and also remain maintainable long-term.
Decision	A **Microservice architecture** is used for the backend system.
Alternatives	*Monolithic Architecture* • *Advantages:* Simpler initial development; easier deployment; lower operational overhead. • *Disadvantages:* Tight coupling; scaling specific components (such as sensor data ingestion and web interfaces) requires scaling the entire application; single point of failure. • *Feasibility:* Poor fit for long-term scalability and diverse IoT workload requirements. *Serverless Architecture* • *Advantages:* No underlying server management; pay-per-execution model; automatic horizontal scaling. • *Disadvantages:* Cold start latency issues; potential vendor lock-in; resources limitation; challenges in managing continuous or long-lived MQTT connections; [31] • *Feasibility:* Weaker fit due to continuous connection and higher cost for high number of execution requirements.
Rationale	Microservices allow independent scaling and deployment of individual components which ensures fault isolation. It also allows for future scalability as the system grows.

Table 7.2: Decision - Communication

Name	**Communication**
Decision	SD-02
Status	*To review*
Problem	Devices used on the farm (sensors, water meters, etc.) use different protocols and data formats to communicate. In this context, the field gateway needs a way to receive readings from all of these devices and to be able to send irrigation commands back to them.
Decision	Our system will use **MQTT** for communication between the deployed farm devices and the field gateway.

Alternatives	*CoAP* • *Advantages:* Smaller overhead due to utilizing UDP for transport; specifically designed for low-bandwidth settings; multicast support. • *Disadvantages:* Reliability is weaker because of UDP; resembles HTTP, communication is one-to-one; less popular. • *Feasibility:* Weaker fit for our requirements. *HTTP/REST* • *Advantages:* Is familiar and universal, has a large variety of client libraries and servers. • *Disadvantages:* Heavy overhead per message; communication is client-initiated, backend cannot push commands to the gateway without polling or other mechanisms; increased bandwidth usage. • *Feasibility:* Poor fit for our context, since polling for commands wastes bandwidth and may cause delays.
Rationale	MQTT is the most relevant option in this case, as it is lightweight and reliable through TCP. It was primarily designed for IoT devices, using publish/subscribe routing to allow for the easy addition or removal of senders/receivers. Additionally, a bi-directional message flow is supported.

Table 7.3: Decision - Data Transfer Pattern

Name	**Data Transfer Pattern**
Decision	SD-03
Status	*To review*
Problem	Farm devices need to transmit their periodic telemetry readings to the field gateway. We must determine whether devices should actively push their data or if the gateway should periodically pull/poll data from them.
Decision	Our system will use a **Push pattern** (via MQTT publish) for data transfer from farm devices to the field gateway.
Alternatives	*Pull pattern* • *Advantages:* The gateway manages data collection and device sleep schedules; simplifying synchronization; great for batch processing. • *Disadvantages:* Must handle duplicated data; results in wasted network bandwidth and battery drainage when devices have no new data; risk of missing data due to scheduling issues; requires scheduling infrastructure. • *Feasibility:* Poor fit; introduces polling latency. *Poll pattern* • *Advantages:* More efficient than pulling; gateway can control frequency of polling; useful for irregular data. • *Disadvantages:* Similar disadvantages as Pull pattern; polling overhead. • *Feasibility:* Poor fit for our requirements.

Rationale	The Push pattern is ideal for our system as field devices can immediately publish readings when available or when thresholds are breached, thus eliminating polling traffic and conserving sensor energy. [11]

In addition to these major software decisions, minor design choices were also made. For instance, the Repository pattern was utilized in the Farm Backend and Regulator Backend to abstract database interactions for PostgreSQL to keep business logic decoupled from direct SQL queries. The Strategy Pattern was used in the Field Gateway to handle diverse message formats and Data Transfer Objects (DTOs) were implemented across the REST API endpoints to encapsulate and decouple incoming sensor payloads and outgoing commands from the database models.

8 — Architecture Evaluation

This chapter evaluates the architecture proposed in previous chapters. The architecture should address the architecturally significant requirements identified in Section 7.1.

8.1 Requirements Verification

The architecturally significant requirements are traced to the elements and decisions that realize them. Section 5.4 provides preliminary evidence for performance, capacity, reliability and availability. The architecture prototype complements this analysis by texting the requirements under controlled conditions.

Requirements	Architectural evidence	Evaluation
FR-1, FR-2, FR-3, FR-6, FR-8, FR-9, FR-10	Field gateway queue managers and protocol translator, farm data provider, weather component, local database and regulator report generator.	Supported by the architecture. FR-1 and FR-9 can additionally be verified through prototype experiments.
FR-4, FR-5	Irrigation planner and authentication component.	Supported by explicit components with dedicated responsibilities.
TNF-1.1, TNF-1.2, TNF-1.3, TNF-1.4	local database, locally deployed backend and database, cached sensor values and notification mechanisms.	Partially verified. Section 5.4 provides an availability estimate. Failure mitigation can be verified through the prototype. The 99% availability target still requires measurements or more detailed component availability data.
TNF-2.1, TNF-2.2, TNF-2.3	Field gateway rate limiting/queueing and the performance and capacity estimates in Section 5.4.	TNF-2.1 is prototype-tested. TNF-2.3 cannot be established from architecture alone and requires validation against historical irrigation results.
TNF-3.1, TNF-3.2	MQTT communication and the protocol translation responsibilities of the field gateway.	Partially satisfied. The abstraction supports interoperability, but compatibility with three manufacturers has not yet been demonstrated.
ER-1, ER-2, ER-3, ER-4	Separate field gateway, weather component, microservice decomposition and local database for storage.	The architecture separates local hardware, external services and irrigation concerns.

Table 8.1: Verification of architecturally significant requirements

There are requirements for which architectural behavior can be observed directly and is there-

fore testable. We can create prototype experiments to verify these testable requirements.

Test	Requirement	Prototype experiment and pass criterion
PVT-1	FR-1	Simulate MQTT sensor publishers and verify that their telemetry reaches the database storage without message loss.
PVT-2	FR-9	Disconnect the backend connection while generating telemetry, then restore the connection and verify that the data has been cached and synchronizes automatically without intervention.
PVT-3	TNF-2.1	Simulate the maximum deployment and the UC1 specified burst load. Record message loss, latency and resource usage. The UC1 persistence target of 100 ms is used as the response-time criterion.
PVT-4	TNF-1.3, TNF-1.4	Disconnect a sensor after a valid measurement. Verify that the next decision can use the last valid value and that the operator is notified within 10 minutes.

Table 8.2: Architecture prototype verification

8.2 Architecture Trade-off Analysis Method

The Architecture Trade-off Analysis Method, or ATAM, is used to investigate the effects of architectural decisions on important quality attributes.

8.2.1 Utility tree

The utility tree is based on the three key drivers identified in Section 3.2. Multiple refinements and corresponding scenarios are included for each key driver. The scenarios are rated by importance (I) for the relevant quality attribute and difficulty of actualization (D) using low (L), medium (M) and high (H).

Key driver	Refinement	Quality-attribute scenario	Priority (I,D)
Performance efficiency	Throughput and latency	**S1:** During peak operation, telemetry from up to 1000 sensors, including the UC1 burst load, is accepted and persisted while maintaining the specified response time.	H, M
Performance efficiency	Resource Utilization	**S2:** When water usage data does not correspond to a performed irrigation decision, the system responds by stopping the automatic implementation of future irrigation decisions. The operator is warned that something may have gone wrong during irrigation	H, L
Reliability	Offline operation	**S3:** Internet connectivity is lost during normal operation. Local data collection, making and implementing irrigation decisions continue. Local and cloud databases are synchronized automatically after connectivity returns.	H, L
Reliability	Fault tolerance	**S4:** A sensor becomes unreachable during normal operation. The system continues using its last valid value and alerts the operator within 10 minutes.	M, L
Reliability	Fault tolerance	**S5:** The database is unresponsive such that reading and writing is not succeeding. The gateway should buffer and cache incoming sensor data until the database responds again.	H, L
Compatibility	Device integration	**S6:** A device from a new manufacturer using a standard IoT protocol is introduced. Support can be added without modifying the Irrigation Planner or interrupting existing operation.	M, H

Table 8.3: ATAM utility tree (precursor)

8.2.2 Scenario Analysis

8.3 Prototype Description

The prototype's primary objective to evaluate and validate how the Farm Backend processes telemetry readings, handles hardware degradation, and outputs appropriate decision logs. It uses a test harness that feeds structured input array to mock the sensors data into the backend system. This setup allows to precisely simulate both normal operating conditions and critical edge cases, such as delayed sensor transmissions, missing data packets, or completely unresponsive field hardware.

Since real-world field data is unavailable at this stage, the prototype operates entirely as an offline simulation. An AI agent - Claude [6], configured via a structured `CLAUDE.md` context file was used to assist with the development of the prototype.

By analyzing the generated decision logs under these simulated conditions, we can verify

the backend's algorithmic correctness and reliability. Specifically, the prototype verifies four system requirements: FR-4, FR-8, TNF-1.3, TNF-2.2.

9 — System Evolution

In previous chapters, we described the baseline features of the proposed Smart Irrigation system, forming a solid foundation. However, as technologies develop and user needs change, the system must keep evolving in order to remain relevant for its target market. As such, this section discusses some future extensions our tool could offer, along with the technical and commercial implications of their implementation.

9.1 Future extensions

9.1.1 Machine learning model for irrigation decisions

The system currently makes irrigation decisions with deterministic logic, using PID control (FR-4) combined with configurable thresholds for soil properties, water volumes and weather readings. Introducing a machine learning (ML) model for this task is the logical next step following the Technology Roadmap, improving the overall precision and efficiency of our system. To fully implement this feature, we foresee three stages:

Initially, the model is not developed in-house. Instead, the system uses a specialized irrigation API that exposes recommendations for a given location and crop type. This stage would require no data science effort, as it is similar to how the Open-Meteo connection was implemented (HDD-02). Several drawbacks arise from using an already-trained model, namely the lack of customization and the dependency on an external provider. As such, the ML output at this stage would not replace any existing logic, serving more as support rather than primary controller. This phase additionally allows us to test user reception to ML-based recommendations before investing in our own model.

In the second stage, we train our own model in the cloud using irrigation decisions that the system has made in the past, together with their measured outcomes. Since single farms only generate a small number of such decisions per growing season, training the model on one site alone would take several years of data gathering. To solve this issue, data from different users can be pooled to build a larger set, also adding to the variety of soil and crop conditions. This, however, introduces concerns linked to GDPR compliance and requires informed user consent. Code-wise, the extension builds on the cloud synchronization of HDD-01: currently only backups are stored in the cloud, so an anonymized training dataset would have to be extracted from them. Since training happens off-site, this stage requires no on-site changes to the system.

In terms of model type, we intend to evaluate tree-based ones such as XGBoost and Random Forest because of their applications in the context of precision irrigation [22]. To mitigate the risk of crop damage, we plan for two crop cycles between the start of data collection and the first model deployment. The first one would be based on collecting training data, while the second would validate the model on pilot farms under the supervision of an agricultural

professional.

For the last phase, the trained model is deployed locally to ensure the system is fully operational even during internet outages (ER-4), being integrated through the Farm Backend service. Since tree-based models are small, running predictions on the ARM-based Central Processing Controller (HD-01) should not exceed the time limits defined in the Non-Functional Requirements. New model versions are delivered through remote updates, while retraining stays in the cloud. From a technical viewpoint, the model replaces the decision logic of the Irrigation Planer. Other components remain unchanged, as the planner communicates only through the Farm Data Provider and the overall microservices are loosely coupled (SD-01). Scheduling remains deterministic, such that hard limits (pump capacity, quotas, safety caps) cannot be overridden by the model.

The cost of this extension consists of one-off development expenses (the data science effort, supervised pilot, and model training) and recurring expenses (cloud provider fees, retraining and monitoring). We estimate the one-off cost at roughly one fifth of the development cost of the baseline system. Since the model is integrated into the system for all customers, the investment is paid off through changes in pricing: the initial selling price could rise by at most 400 € (20%), and the per-hectare annual subscription by at most 25% (from 40 to 50 € per hectare).

We only discuss financial implications for this extension in particular, because it entails significant costs beyond regular software development, which needed to be reflected in the final product pricing.

9.1.2 Awareness of available energy resources

Our hardware architecture assumes a hybrid power source that combines both grid and solar energy (HD-06), but the software has no knowledge of available power and its cost. A possible extension would make the system aware of both, using this information to better schedule pumps and valves, reducing overall operating costs as well as lower the farm's carbon footprint. This change requires a meter or inverter that reports energy production and consumption. Such devices typically communicate over protocols like Modbus instead of MQTT, so integrating them requires a protocol adapter in the Field Gateway, which already translates between device protocols and the backend (SD-02, ER-01, TNF-4.1). Existing logic would change only for scheduling, thus the Irrigation Planner would need to be extended.

9.1.3 Crop health and yield insights

The system currently only focuses on irrigation by measuring soil conditions and water amounts, with crop health and expected yield only being addressed indirectly. An meaningful extension estimates both for each available crop type and shows the results to the owner, giving them concrete figures to base financial decisions on, such as expected harvest volumes or fields where the crop is under stress. This can be provided through the ML model proposed earlier, but a simpler deterministic estimate of yield and crop health could be delivered earlier. In both cases, the Irrigation Planner is extended to compute the estimates, crop parameters are added as data, and the Dashboard and Historical Data components present the additional information. In terms of design decisions, none of the existing ones need to change, as this change builds on data and components that are already there.

9.1.4 Multiple farm locations per owner

Because we are targeting medium to large scale businesses, some owners may run farms in several locations. For this case specifically, viewing aggregated data at client level would be a valuable addition. This extension requires the cloud database to become the aggregation point, given the unreliable on-site internet access. Since the baseline architecture the cloud only holds backups (HDD-01), it would need to become a queryable service. The data models would require the addition of a farm/site identifier and the Owner Dashboard would need a multi-site view.

9.2 Versions

Table 9.1 shows the upcoming versions of the Smart Irrigation system, each version implementing one of the extensions detailed in Section 9.

Version	Description
1.0	Baseline features as described in Sections 1-8.
1.1	Crop health and yield insights.
1.2	Awareness of available energy resources.
1.3	First phase of ML model integration for irrigation decisions.
2.0	Multiple farm locations per owner.
2.1	Second phase of ML model integration for irrigation decisions.
3.0	Third phase of ML model integration for irrigation decisions.

Table 9.1: Planned versions of the Smart Irrigation system

In this document, we detailed the architecture of a Smart Irrigation system intended for the Mediterranean region. Its design was centered around reducing water consumption, ensuring reliable operation, and being hardware-agnostic. The resulting system is a local-first, microservice-based application that optimizes water usage through a closed loop deterministic logic. It remains operational even in the event of internet outages, and its cloud and local database backups protect the collected data against loss. Building on this foundation, the extensions discussed in this chapter can be added without a full redesign, shaping the system into a more robust solution for water-efficient farming.

References

[1] ISO 25000. *ISO/IEC 25010*. URL: https://iso25000.com/index.php/en/iso-25000-standards/iso-25010.

[2] K. S. Adu-Manu et al. "Innovations in Sensor-Based Systems and Sustainable Energy Harvesting in Precision Agriculture". In: *Sensors* 23.12 (2023), p. 5678.

[3] Eclipse Foundation AISBL. *Mosquitto*. URL: https://mosquitto.org/.

[4] Inc. Amazon Web Services. *Amazon Web Services*. URL: https://aws.amazon.com/.

[5] Inc. Amazon Web Services. *Data protection in Amazon S3*. 2026. URL: https://docs.aws.amazon.com/AmazonS3/latest/userguide/DataDurability.html.

[6] Anthropic. *Claude*. 2026. URL: https://claude.ai.

[7] Ferenc Badia. *Irrigation Infrastructure Digitalization Project in Sardinia*. 2024. URL: https://www.bermad.com/kc/case-studies/irrigation-infrastructure-digitalization-sardinia/.

[8] European Commission. *Increasing the resilience of agriculture in water and nutrient-scarce environments through digital innovations*. 2026. URL: https://ec.europa.eu/info/funding-tenders/opportunities/portal/screen/opportunities/topic-details/HORIZON-CL6-2027-02-FARM2FORK-01.

[9] ControlByWeb. *Traditional PLCs vs Edge Controllers: What's the Difference?* Accessed: 2026-10-01. Apr. 2024. URL: https://controlbyweb.com/blog/traditional-plcs-vs-edge-controllers/.

[10] Yassmin H. Essa et al. "Drought characteristics in Mediterranean under future climate change". In: *npj Climate and Atmospheric Science* (2023). DOI: 10.1038/s41612-023-00458-4.

[11] Ime Eti-mfon. *Comparing Data Transfer Patterns*. https://medium.com/@etimfonime/comparing-data-transfer-patterns-291b694ee106. Accessed: 2026-10-15. Dec. 2025.

[12] Market Data Forecast. *Europe Intelligent Agriculture Market Size, Share, Trends & Growth Forecast Report, Segmented By Agriculture, Application, Offering and Country (The UK, France, Spain, Germany, Italy, Russia, Sweden, Denmark, Switzerland, Netherlands, Turkey, Czech Republic and Rest of Europe), Industry Analysis From 2026 to 2034*. 2026. URL: https://www.marketdataforecast.com/market-reports/europe-intelligent-agriculture-market.

[13] Apache Software Foundation. *Kafka*. URL: https://kafka.apache.org/.

[14] Cloud Native Computing Foundation. *Kubernetes*. URL: https://kubernetes.io/.

[15] OpenJS Foundation. *Node.js*. URL: https://nodejs.org/en.

[16] Python Software Foundation. *Python*. URL: https://www.python.org/.

[17] Google. *Angular*. URL: https://angular.dev/.

[18] Google. *Google Cloud*. URL: https://cloud.google.com/.

[19] Italian Government. *Investments in agro-irrigation system resilience for better water resource management*. URL: https://www.italiadomani.gov.it/content/sogei-ng/it/en/Interventi/investimenti/investimenti-nella-resilienza-dell-agro-sistema-irriguo-per-una-migliore-gestione-delle-risorse-idriche.html.

[20] The PostgreSQL Global Development Group. *PostgreSQL*. URL: https://www.postgresql.org/.

[21] Docker Inc. *Docker*. URL: https://www.docker.com/.

[22] Saviour Inyang, Daniel Essien, and Ezichi Mpamugo. "Comparative Analysis of Machine Learning Models for Irrigation Technique Classification in Precision Agriculture". In: *International Journal of Applied Information Systems* 12 (June 2025), pp. 37–46. DOI: 10.5120/ijais2025452020.

[23] Philippe Kruchten. "Architectural Blueprints—The "4+1" View Model of Software Architecture". In: *IEEE Software* 12.6 (1995), pp. 42–50. DOI: 10.1109/52.469759.

[24] Inc Meta Platforms. *React*. URL: https://react.dev/.

[25] Microsoft. *Microsoft Azure*. URL: https://azure.microsoft.com/en-us.

[26] MQTT.org. *MQTT*. URL: https://mqtt.org/.

[27] T. Ojha, S. Misra, and N. S. Raghuwanshi. "Review of IoT and electronics enabled smart agriculture: Networks, architectures, and applications". In: *International Journal of Agricultural and Biological Engineering* 11.4 (2018), pp. 1–11.

[28] M. Oojannah and K. K. Khedo. "Energy-Efficient Wireless Sensor Networks for Precision Agriculture: A Review". In: *Sensors* 17.8 (2017), p. 1781. DOI: 10.3390/s17081781.

[29] Sharaf Al-Otaibi, Mohammad Al-Maitah, and Hazem Al-Rawajfeh. "A Secure IoT-Based Irrigation System for Precision Agriculture Using Lightweight Cryptography". In: *Sensors (Basel)* 23.4 (2023), p. 2012. DOI: 10.3390/s23042012.

[30] T. Letterio P. Mannini R. Genovesi. "IRRINET: Large Scale DSS Application for On-farm Irrigation Scheduling". In: *Procedia Environmental Sciences* 19 (2013), pp. 823–829. DOI: 10.1016/j.proenv.2013.06.091.

[31] Dave Patten. *Serverless Architecture: Pros, Cons, and Use Cases*. https://medium.com/@dave-patten/serverless-architecture-pros-cons-and-use-cases-4a0769744ca2. Accessed: 2026-10-15. Nov. 2024.

[32] Pallavi Premkumar. *European Developer Hourly Rates in 2026: Full Cost Guide by Region*. 2025. URL: https://www.index.dev/blog/european-developer-hourly-rates.

[33] PRIMA. *Integrated and sustainable management of water for arid and semi-arid Mediterranean areas*. 2026. URL: https://prima-med.org/.

[34] Canale Emiliano Romagnolo. *Irriframe - Il portale dell'irrigazione*. URL: https://www.irriframe.it/irriframe.

[35] S. Seneviratne et al. "Renewable Energy Harvesting for Wireless Sensor Networks in Precision Agriculture". In: *IEEE Transactions on Industrial Informatics* 16.9 (2020), pp. 5920–5931.

[36] Italian National Institute of Statistics (ISTAT). *Struttura produttiva, pratiche gestionali e forza lavoro delle aziende agricole*. 2026. URL: https://www.istat.it/wp-content/uploads/2026/07/Focus-Struttura-produttiva-pratiche-gestionali-e-forza-lavoro-delle-aziende-agricole_Anno-2023.pdf.

[37] Italian National Institute of Statistics (ISTAT). *Water statistics*. 2026. URL: https://www.istat.it/wp-content/uploads/2026/06/Istat_WWD2026_Focus.pdf.

[38] Inc. The Channel Company. *Cloud Market Share Q2 2026: Google Gains Share As AWS Falls*. 2026. URL: https://www.crn.com/news/cloud/2026/cloud-market-share-q2-2026-google-gains-share-as-aws-falls.

[39] University of Minnesota Extension. *Soil moisture sensors for irrigation scheduling*. https://extension.umn.edu/natural-resources/conservation/agricultural-soil-and-water/irrigation/soil-moisture-sensors-for-irrigation-scheduling. Accessed: 2026-09-26.

[40] Nikhil D. Shankar Walter P. Falcon Rosamond L. Naylor. "Rethinking Global Food Demand for 2050". In: *Population and Development Review* 48.4 (2022), pp. 921–957. DOI: 10.1111/padr.12508.

[41] Wikipedia. *MoSCoW method*. URL: https://en.wikipedia.org/wiki/MoSCoW_method.

[42] Wikipedia. *SWOT analysis*. URL: https://en.wikipedia.org/wiki/SWOT_analysis.

[43] Evan You. *Vue*. URL: https://vuejs.org/.

A — Architecture Decision Records

A.1 Hardware Decisions

A.1.1 Controller Architecture

Name	**Controller Architecture**
Decision	HD-01
Status	*To review*
Problem	The smart irrigation system requires a robust processing unit to collect soil data, calculate irrigation schedules and control solenoid valves across fields.
Decision	Deploy an ARM-based Central Processing Controller equipped with local processing capabilities, direct API integration, Wi-Fi, and scalable Ethernet connectivity to accommodate an expanding network of hardware Field Data Relay connections.
Alternatives	*Traditional Industrial PLC (Programmable Logic Controller)* • *Advantages:* High reliability, industrial-grade ruggedness (IP67), long operational lifespan, and deterministic real-time valve control. • *Disadvantages:* High hardware and software licensing costs; limited native support for modern IoT protocols (MQTT, HTTPS APIs); rigid proprietary programming (e.g., Ladder Logic); lacks local capacity to run predictive dynamic algorithms. • *Feasibility:* Medium. Overly complex, rigid, and cost-prohibitive for adaptive smart irrigation algorithms. *Direct Cloud-Connected Microcontroller (No Edge Layer)* • *Advantages:* Very low unit hardware cost per controller board; simplified field hardware setup. • *Disadvantages:* Total reliance on continuous internet connectivity; operational latency; high risk of water loss or crop damage during network outages; increased cloud data bandwidth costs. • *Feasibility:* Low. Unreliable for rural locations prone to intermittent network coverage.
Rationale	A Central Processing Controller with expandable Field Data Relay network offers localized offline processing (preventing failure during network loss), seamless integration with external weather APIs and significantly lower cost-to-complexity overhead compared to traditional PLCs. [9]

Table A.1: Decision - Controller Architecture

A.1.2 Soil Moisture Sensor

Name	**Soil Moisture Sensor**
Decision	HD-02
Status	*To review*
Problem	The smart irrigation system requires accurate soil moisture monitoring to prevent over- and under-irrigation.
Decision	Integrate a Volumetric Water Content (VWC) Capacitance Sensor (Truebner SMT100) to monitor soil moisture levels across fields.
Alternatives	*Soil Water Tension Sensor (Irrometer Tensiometers)* • *Advantages:* Lower sensor unit cost ($80 per sensor; $140–$155 for transducers); directly measures the energy required by plant roots to extract water. • *Disadvantages:* Highly soil-specific measurements which can be inaccurate, thus requiring constant calibration; poor compatibility with automated systems. • *Feasibility:* Medium. High maintenance overhead. *Time Domain Reflectometry (TDR) Sensors* • *Advantages:* Very high accuracy across all soil types with low measurement error; highly resistant to environmental variables. • *Disadvantages:* Significantly higher upfront cost ($250–$350 per sensor) with costly data loggers ($500–$2,500); higher energy consumption and complex maintenance. • *Feasibility:* Low. High equipment costs.
Rationale	Capacitance sensors like the Truebner SMT100 offer minimal maintenance, high automation compatibility, and low energy consumption while delivering dependable VWC measurements. [39]

Table A.2: Decision - Soil Moisture Sensor

A.1.3 Flow meter

Name	**Flow meter**
Decision	HD-03
Status	*To review*
Problem	Water & fertilizer quantities used in irrigation need to be measured using flow meters. These devices have to remain accurate and functional despite harsh climate conditions and impurities found in the water.
Decision	Given the existing climate concerns, the best fit for our system would be the Seametrics AG3000 Series Flanged Magmeter.
Alternatives	*Octave - Stainless Steel Floating Flanges Ultrasonic Water Meter* • *Advantages:* Uses ultrasonic technology, claims to be extremely sensitive and accurate even for low flows; caters towards aggressive water environments; includes a data logger and is compatible with a range of protocols. • *Disadvantages:* Is battery powered, claiming to have up to 15 years of life time, but that may not hold true for Mediterranean climate conditions; its recommended ambient temperature is capped at $55°C$, which may be too little. • *Feasibility:* Ultrasonic devices are usually the most expensive and require professional installations. *McCrometer - McPropeller* • *Advantages:* Claims to be especially suited for dirty water flows and high-volume water applications, and does not need external power or batteries. • *Disadvantages:* Its accuracy is moderate compared to other existing flow meter types ($\pm$ 2%); needs periodic servicing due to the main propeller component. • *Feasibility:* This device is said to be economical, but requires more maintenance than other types.
Rationale	The *AG3000 Series Flanged Magmeter* from *Seametrics* fits our overall requirements: it's an electromagnetic flow meter that is resistant to impurities found in water, it can withstand temperatures of up to $60°C$ and its accuracy sits at around $\pm$ 1%. One downside would be that it uses batteries, but this is ameliorated by the fact that the first replacement is free. It also offers a 5 year warranty.

Table A.3: Decision - Flow meter

A.1.4 Solenoid Valve

Name	**Solenoid Valve**
Decision	HD-04
Status	*To review*
Problem	Water and fertilizer flow should be controlled through a solenoid valve, such that no resources are wasted when automatic irrigation is not enabled. As was the case for flow meters, the valve should be resistant to Mediterranean-specific climate conditions.
Decision	Given the existing climate concerns, the best fit for our system would be the Bermad IR-410-3W-RX.
Alternatives	*Gould - General Purpose Type K & KR Series* • *Advantages:* Highly customizable in terms of voltage and pressure range and other factors; is watertight/corrosive resistant; the solenoid is included. • *Disadvantages:* Made for general purpose applications, not specifically irrigation in large-scale agriculture; pipe sizes are capped at DN50, which alone is not enough for large-scale farming. • *Feasibility:* Quite expensive; to make up for its small size, multiple pipes would have to be installed. *Netafim - 100 Series Metal Control Valves* • *Advantages:* Used especially when large diameters are needed; it's reinforced, such that it can be used on field crops, vineyards, and orchards; various options for the solenoid. • *Disadvantages:* The solenoid is sold separately; the valve has a recommended minimum flow (1 m^3/h), which may be troublesome during seasons with high water scarcity; it is built for high pressure applications, which may be out of scope. • *Feasibility:* The device that cannot be fully taken advantage of due to climate-related factors.
Rationale	The *Bermad IR-410-3W-RX* fits our overall requirements: it is intended specifically for irrigation, covers a considerable size range (DN25-DN300) and operates on water pressures starting at 0.5 bar. In contrast to the Netafim product, their documentation lists Low Supplied Pressure Irrigation Systems as one of its applications. Lastly, the product claims to be damage resistant, and its solenoid S-400-3W-PB suits a typical controller.

Table A.4: Decision - Solenoid Valve

A.1.5 Connectivity

Name	**Connectivity**
Decision	HD-05
Status	*To review*
Problem	The system requires reliable, low-latency communication between field sensors, physical controllers, gateways, and cloud/local servers.
Decision	Adopt a Hybrid Network Architecture utilizing a low-power Zigbee mesh for local sensor-to-gateway telemetry and Wi-Fi for gateway-to-cloud communication.
Alternatives	*Low-Power Wide-Area Network (LPWAN - LoRaWAN)* • *Advantages:* Exceptional range (10–15 km); ultra-low power consumption; well-suited for widely dispersed open fields. • *Disadvantages:* Very low data throughput; requires setting up local gateways or paying recurring cellular SIM subscriptions per node. • *Feasibility:* Medium. Overkill for dense, localized crop clusters; higher subscription/hardware overhead. *Wi-Fi Network* • *Advantages:* High bandwidth; seamless cloud integration; direct connection to local routers. • *Disadvantages:* High power consumption rapidly drains battery-operated field sensors; limited transmission range (30–100 m) requiring extensive infrastructure. • *Feasibility:* Low. Impractical for battery-powered edge nodes. *Zigbee Network* • *Advantages:* Low power usage; low latency for localized node-to-node routing and reliable mesh networking. • *Disadvantages:* Limited per-hop range (10–100 m); lacks native IP/cloud connectivity without an external internet gateway. • *Feasibility:* Medium. Incomplete without internet gateway.
Rationale	The Hybrid (Zigbee + Wi-Fi) approach minimizes power usage at remote sensor nodes while maintaining robust local mesh reliability. [28, 27]

Table A.5: Decision - Connectivity

A.1.6 Power Supply

Name	**Power Supply**
Decision	HD-06
Status	*To review*
Problem	The system must deliver continuous, reliable electrical power to various field sensors and actuators (solenoid valves and pumps).
Decision	Implement a Hybrid Power Architecture combining Solar Photovoltaic (PV) with Mains AC grid power.
Alternatives	*Mains AC / Grid Power Only* • *Advantages:* High power capacity; stable 110V–240V supply; capable of running heavy pumps and controllers continuously. • *Disadvantages:* Infeasible for remote field sensor nodes far from farm electrical grid infrastructure; high wiring installation costs. • *Feasibility:* Low for field nodes; High for central pump stations. *Primary (Non-Rechargeable) Batteries* • *Advantages:* Low initial hardware cost; compact form factor; high energy density with minimal self-discharge. • *Disadvantages:* High long-term operational maintenance cost due to periodic replacements; continuous hazardous battery wastage. • *Feasibility:* Low. Unsuitable for long-term automated operation. *Solar PV + Rechargeable Battery Only* • *Advantages:* Off-grid self-sustainability for remote sensor deployment; eliminates battery replacements. • *Disadvantages:* Weather-dependent generation; insufficient sustained current output for heavy-duty water pumps. • *Feasibility:* Medium. Excellent for remote edge nodes, inadequate for primary pump actuation.
Rationale	The hybrid power approach provides off-grid solar energy for remote field sensors and grid power for heavy pumps, eliminating field battery maintenance while ensuring continuous operation during power outages. [2, 35]

Table A.6: Decision - Power Supply

A.1.7 Water Pump

Name	**Water Pump**
Decision	HD-07
Status	*To review*
Problem	Water has to be gathered and distributed over large agricultural fields using water pumps for irrigation purposes. These devices have to be effective and resilient given climate conditions.
Decision	Given the existing climate concerns, the best fit for our system would be a Submersible Multistage Pump.
Alternatives	*High-Flow Centrifugal Pump* • *Advantages:* Handles high water flows with precision and speed; has good durability; can run on a variety of power sources. • *Disadvantages:* Needs a surface water stream to pump resources which are then distributed across the fields; struggles with deeper water sources. • *Feasibility:* Requires less maintenance; generally less costly. *Vertical Turbine Pump* • *Advantages:* Best choice for large-diameter wells and very high water volumes; motor is placed on the surface; can run on a variety of power sources. • *Disadvantages:* Needs a straight and plumb well for its shaft; depth limit caps at 150 m/200 m because of shaft length, stretch, and weight. • *Feasibility:* Complex installation and servicing overall.
Rationale	A Submersible Multistage pump would be the best decision, since it has no limit for depth and can be used in crooked, narrow wells. Most such pumps are solar-powered, meaning they are autonomous and have overall low operating costs once installed. Maintenance may be inconvenient since the motor is placed underwater, below the bowls.

Table A.7: Decision - Water pump

A.2 Title

Change Log

Date	Status
2024-10-04	proposed
2024-10-14	superseded by A.3

What is the status, such as proposed, accepted, rejected, deprecated, superseded, etc.?

Context

What is the *issue* that we're seeing that is motivating this decision or change?

Point to the relevant use case or user story.

Considerations

What are the alternatives? What are the forces?

Decision

What is the change that we're proposing and/or doing?

Consequences

What becomes easier or more difficult to do because of this change?

Related Decisions

What other decisions are affected by or influence this decision? Elaborate.

A.3 Title

Change Log

Date	Status
2024-10-14	proposed (superseded A.2)

What is the status, such as proposed, accepted, rejected, deprecated, superseded, etc.?

Context

What is the *issue* that we're seeing that is motivating this decision or change?

Point to the relevant use case or user story.

Considerations

What are the alternatives? What are the forces?

Decision

What is the change that we're proposing and/or doing?

Consequences

What becomes easier or more difficult to do because of this change?

Related Decisions

What other decisions are affected by or influence this decision? Elaborate.

B — Time Tracking

B.1 Week 1

Person	Date	Task	Hours
dr	2026-09-01	Setting up the document	1
dr, mv, ids, aknh	2026-09-02	Team meeting - discussing system context & business vision	1
dr	2026-09-03	Drafting 2.1 and 2.8 sections	2
mv	2026-09-03	Drafting section 2.3 & SWOT analysis	2
jk	2026-09-03	Drafting section 3 & Review	2
aknh	2026-09-03	Drafting section 1 and 2.6	1.5
ids	2026-09-03	Drafting section 3.3	2
all	2026-09-04	Team meeting with coach	1
		Total - dr	5
		Total - mv	4
		Total - jk	3
		Total - aknh	3.5
		Total - ids	4
		Group Total	19.5

B.2 Week 2

Person	Date	Task	Hours
all	2026-09-07	Team meeting - narrowing down the scope	1
all	2026-09-08	Research discussion	0.5
jk	2026-09-10	Update stakeholders based on design workout 1	2
dr	2026-09-10	Update competitors based on the scope	1
dr	2026-09-10	Add domain and business models	1.5
mv	2026-09-10	Add Business Rationale, updated Business Vision and Target Audience	2.5
aknh	2026-09-10	Refine context and solution description narrowing down project scope	0.75
aknh	2026-09-10	Add product development business strategy in Business Model and Update Financial Model	1
ids	2026-09-10	Refining section 3.3	3
all	2026-09-11	Team meeting with Coach	1
all	2026-09-11	Team meeting discussing requirements	0.5
		Total - dr	5.5
		Total - mv	5.5
		Total - jk	5
		Total - aknh	4.75
		Total - ids	6
		Group Total	26.75

B.3 Week 3

Person	Date	Task	Hours
ids	2026-09-14	Adding business roadmap	1
dr	2026-09-14	Added assumptions and descriptions for figures 2.1 & 2.2	1.5
jk	2026-09-14	Additional background research and requirements drafting	2
ids	2026-09-15	Completed financial model	2
all	2026-09-15	Team meeting - discussing design decisions & software architecture	0.5
mv	2026-09-15	Added product description and non-functional requirements	3
jk	2026-09-15	Added evolution requirements	1
all	2026-09-15	Design Workout week 3	2
ids	2026-09-16	Added technology roadmap	0.5
dr	2026-09-16	Added functional requirements	1.5
dr	2026-09-16	Added high-level design decisions	2
mv	2026-09-16	Set up & worked on the slides for chapters 1 and 2 for the first presentation	1.5
ids	2026-09-17	Sketched C4 context and containers views	1
jk	2026-09-16	Added ISO/IEC 25010 description and document review	1
aknh	2026-09-17	Added Architectural Vision	0.75
aknh	2026-09-17	Refined Use-case scenarios 1 to 3 & Fixed sections 3.1 to 3.3 formating	1.5
aknh	2026-09-17	Added three risks and Risk Assessment diagram	1.5
all	2026-09-18	Team meeting with Coach	1
dr	2026-09-18	Updated figures 2.1 & 2.2	0.25
jk	2026-09-18	Fixed minor formatting and styling issues across multiple sections	0.5
ids	2026-09-18	Refined C4 models and descriptions	0.5
aknh	2026-09-18	Refined Use-case scenario 4 & Updated Risk Assessment with 3 additional risks	1
		Total - dr	8.75
		Total - mv	8
		Total - jk	8
		Total - aknh	8.25
		Total - ids	8.5
		Group Total	41.5

B.4 Week 4

Person	Date	Task	Hours
ids	2026-09-19	Presentation's slide: stakeholders and architecture	1
jk	2026-09-20	Group 1 document review & preparing presentation questions	1.5
mv	2026-09-20	Refactored my half of the presentation slides, rehearsed presentation speech	1.5
ids, mv, dr, aknh	2026-09-22	Design Workout - Haiku	2
ids, mv, dr	2026-09-22	Team meeting - discussing chapter 5,6,7	0.25
dr	2026-09-23	Drafted 7.1 hardware components and group 1 feedback	1.5
jk	2026-09-23	Group 1 Feedback document	1
jk	2026-09-24	Adding section 5.3	1.5
mv	2026-09-24	Group 1 Feedback & section 7.1	1
aknh	2026-09-24	Group 1 Feedback	1
aknh	2026-09-24	Refined Architecture Vision & Added details for Risk Table	0.33
aknh	2026-09-24	Modified UC5 and UC6 & Added failing scenario for UC1	1.5
ids	2026-09-24	Refined tech roadmap, section 5.1 and 5.2	2
all	2026-09-25	Team meeting with Coach	0.5
ids, mv, dr, jk	2026-09-25	Team meeting - discussing feedback	0.5
		Total - dr	4.75
		Total - mv	5.75
		Total - jk	5
		Total - aknh	3.83
		Total - ids	6.25
		Group Total	26.08

B.5 Week 5

Person	Date	Task	Hours
ids, mv, dr, jk	2026-09-29	Design Workout - Decision Cards	2
dr	2026-10-01	Added Hardware Decisions (HD-01, HD-02)	3
aknh	2026-10-01	Added hyperlinks for Functional Requirements in use-case scenarios table	0.33
mv	2026-10-01	Added Hardware Decisions HD-03, HD-04	3.5
ids	2026-10-01	Detailed 5.2 diagram and technologies rationale	3.5
jk	2026-10-01	Fixed formatting, rewrote 5.4 and investigated writing future sections	1.5
all	2026-10-02	Team meeting with Coach	0.5
all	2026-10-02	Team meeting - feedback review & task distribution	0.5
		Total - dr	6
		Total - mv	6.5
		Total - jk	4.5
		Total - aknh	1.33
		Total - ids	6.5
		Group Total	23.33

B.6 Week 6

Person	Date	Task	Hours
ids	2026-10-02	Provided logo and model alternatives in 5.3	1
jk	2026-10-05	Document review	2
dr	2026-10-06	Added Hardware Decisions (HD-05, HD-06)	3
mv	2026-10-06	Added Hardware Decisions (HD-07, HD-08)	2
ids	2026-10-06	Document review	1.5
all	2026-10-06	Team meeting - entire document review and chapters 6-7	2
jk	2026-10-07	Corrections and comments based on meeting	1
ids	2026-10-07	Drafted regulator backend logical view	1
ids	2026-10-07	Drafted deployment view	1
mv	2026-10-08	Drafted farm backend logical view	2.5
dr	2026-10-08	Drafted user interface logical view	2
ids	2026-10-08	Detailed sections 5.1 and 5.2	2.5
dr	2026-10-09	Fix corrections based on comments	0.5
all	2026-10-09	Team meeting with coach	0.5
		Total - dr	8
		Total - mv	7
		Total - jk	5.5
		Total - aknh	2.5
		Total - ids	9.5
		Group Total	32.5

B.7 Week 7

Person	Date	Task	Hours
ids	2026-10-12	Review and minor updates chapter 1-2	1
ids	2026-10-13	Update business roadmap	1
all	2026-10-13	Design Workout - System Evaluation	2
all	2026-10-13	Team meeting - task distribution & presentation discussion	0.25
dr	2026-10-13	Moved hardware decisions to appendix & 7.2.5 Field Gateway Components	2
ids	2026-10-13	Reworked financial model	1
mv	2026-10-13	Refined business vision & product description & solved other minor comments	0.5
ids	2026-10-14	General adjustments and refined tech roadmap	2
dr	2026-10-14	Added software design decisions	2
jk	2026-10-15	Added Architecture Evaluation	2
mv	2026-10-15	Added Database Schema & started implementation view	1.5
ids	2026-10-15	Refine deployment view and draft process view	2
aknh	2026-10-15	Refined Hardware Decision HD-1 & Added Hardware Overview description and diagram	2.5
mv	2026-10-16	Added implementation view (includes time spent on diagrams that didn't make the final cut)	2
aknh	2026-10-16	Refined Use cases UC-2, UC-3 and UC-5 & Updated Use case diagram	2.25
aknh	2026-10-16	Updated Risk Assessment R4 and R5	0.25
all	2026-10-16	Team meeting with Coach	0.5
		Total - dr	6.75
		Total - mv	6.75
		Total - jk	4.75
		Total - aknh	7.75
		Total - ids	9.75
		Group Total	35.75

B.8 Week 8

Person	Date	Task	Hours
dr	2026-10-17	Second presentation slides and preparation	2
dr	2026-10-19	Group 2 document review and feedback	0.5
all	2026-10-20	Design workout - prototype	2
mv	2026-10-22	Added chapter 9	3.5
dr	2026-10-22	Added prototype description	0.5
ids	2026-10-22	Update deployment and process diagrams	3
dr, ids, mv, aknh	2026-10-23	Team meeting with Coach	0.5
		Total - dr	6
		Total - mv	6.5
		Total - jk	2.5
		Total - aknh	3
		Total - ids	6
		Group Total	24

B.9 Week 9

Person	Date	Task	Hours
ids	2026-10-26	Refined process view	1.5
dr	2026-10-27	Refine logical view diagrams	1
mv	2026-10-27	Replaced c4 mentions & refined chapter 5 diagrams. Added hyperlinks and conclusion to chapter 9.	2
ids	2026-10-27	Refined alternative models	1
		Total - dr	1
		Total - mv	2
		Total - jk	0
		Total - aknh	0
		Total - ids	2.5
		Group Total	5.5